

Progetto Sistemi Operativi - Ing. Tor Vergata

Battaglia Navale Multiutente

([Repository GitHub](#))

Relazione Tecnica

Realizzato da:

Lorenzo Tarantino (0356183)

Leonardo Rocco Cicalini (0358505)

Corso Sistemi Operativi
A.A 2025-2026

Specifica di progetto.....	3
Descrizione generale.....	3
Server.....	3
Startup del server.....	4
Avvio e configurazione delle strutture dati.....	4
Gestione dei segnali.....	4
Discovery UDP.....	5
La Lobby.....	5
Sincronizzazione tra i player.....	6
Client_thread.....	7
Logica di gioco.....	7
End Game e chiusura.....	8
Protocollo di comunicazione.....	8
Bot come nuovo player.....	9
Client.....	9
Startup del client.....	10
Differenze tra POSIX e WINAPI.....	10
Connessione al server.....	10
Ricezione informazioni utente e inizializzazione griglia.....	11
Gameplay.....	12
Interfaccia Utente.....	12
Chiusura.....	13
Bot.....	13
Startup del Bot.....	13
Piazzamento delle navi.....	13
Selezione della mossa.....	13
Validazione scelta delle celle.....	14
Codice sorgente ed esecuzione.....	14
Manuale d'uso.....	14
Prerequisiti.....	14
La compilazione.....	14
Esecuzione.....	15
Server.....	15
Client.....	15
server.c.....	15
client.c.....	36
bot.c.....	47
gui.h.....	54
gui.c.....	55
protocollo.h.....	64
protocollo.c.....	66
utils.h.....	66
utils.c.....	67
Makefile.....	70

Specifica di progetto

“Realizzazione di una versione elettronica del famoso gioco "battaglia navale" con un numero di giocatori arbitrario. In questa versione più processi client (residenti in generale su macchine diverse) sono l'interfaccia tra i giocatori e il server (residente in generale su una macchina separata dai client). Un client, una volta abilitato dal server, accetta come input una mossa, la trasmette al server, e riceve la risposta dal server. In questa versione della battaglia navale una mossa consiste oltre alle due coordinate anche nell'identificativo del giocatore contro cui si vuole far fuoco. Il server a sua volta quando riceve una mossa, comunica ai client se qualcuno è stato colpito se uno dei giocatori è il vincitore (o se è stato eliminato), altrimenti abilita il prossimo client a spedire una mossa. La generazione della posizione delle navi per ogni client è lasciata alla discrezione dello studente.”

Descrizione generale

I programmi sviluppati si occupano di gestire il funzionamento del gioco “battaglia navale” che in questa versione è presentata nella variante multiplayer. Il progetto si basa su un'**architettura software server-client**: sviluppato per piattaforma mista UNIX/WINAPI, il server è realizzato per standard POSIX mentre il client è cross-platform così che le due applicazioni possano risiedere su macchine di natura diversa e comunicare tramite i socket. Il server permette di accettare le richieste di connessione da nuovi player (client), orchestra e controlla la partita in corso e valida le mosse in arrivo. Inoltre, l'architettura del server implementa un meccanismo di *fallback* in fase di startup: qualora la porta inserita in fase di esecuzione non sia valida o occupata, si fa carico di trovarne una disponibile e di comunicarla ai client. Oltre ai codici per il server e per il client richiesti ve n'è un altro: il codice per il funzionamento del bot (attivato solamente quando si ha una sola connessione accettata dalla lobby). Lo sviluppo ed i test sono stati effettuati eseguendo client e server sulla stessa rete locale, utilizzando più VM. Il test per la connessione su reti diverse è stato effettuato tramite VPN (ZeroTier) verificando entrambe le modalità: automatica tramite discovery, e manuale inserendo l'IP della macchina che ospita il server e la porta su cui è in ascolto.

Server

Il codice del server è interamente **realizzato per standard POSIX**. In fase di rilascio, generalmente, il server rimane indipendente dalle macchine che eseguiranno lo script dei client, per questo motivo il server è esclusiva standard POSIX, mentre i **client**, come verrà spiegato nella sezione apposita, sono **cross-platform**.

Il server accetta connessioni sia da macchine che seguono standard WINAPI, sia POSIX, questo grazie ai socket di comunicazione in standard `AF_INET` e alla realizzazione di un protocollo di comunicazione apposito. Il codice è **multithread**: ad ogni client connesso viene assegnato un thread di gestione apposito. Così facendo si garantiscono performance maggiori e minor occupazione di memoria e utilizzo di risorse. Tuttavia, l'esecuzione di codice in ambiente multithread impone una corretta gestione e prevenzione delle race condition. I problemi di sincronizzazione qui vengono risolti tramite l'utilizzo di mutex (nella struttura `gstate` che serve per il controllo della partita vi è un mutex locale che protegge

l'accesso alle risorse) e semafori, questi ultimi garantiscono sincronia durante i turni nel gameplay. Il server utilizza due insiemi di semafori tra loro distinti: `sem1` serve per la gestione dei turni tra client con dimensione pari a `MAX_PLAYER`; `sem2` agisce invece da barriera, bloccando i client (quindi l'avvio della partita) fino a quando tutti i client hanno completato il posizionamento, tramite delle post (semaforo ad **uno slot** inizializzato a 0 sul quale il main si blocca con `sem_op` a `-slot`, sbloccandosi dopo aver aspettato tutti gli `slot` gettoni).

Startup del server

Per poter avviare il server va digitato a riga di comando `./battaglia_server <port>` dopo aver eseguito correttamente il comando di compilazione `make server` del Makefile. In questo modo si può avviare il server che eseguirà i controlli sui dati di input (quindi in questo caso la porta) e avvierà l'installazione delle strutture dati necessarie alla sincronizzazione e alla gestione delle segnalazioni, aprendo poi i socket per la comunicazione.

Avvio e configurazione delle strutture dati

All'avvio del server vengono eseguiti gli usuali controlli di plausibilità dei dati in ingresso (`<port>`). La porta viene validata **controllando se appartiene al range 5000-65535 (server port non-privileged)**. Qualora non sia una porta valida, il server ne genera una casuale nel range delle porte disponibili. Successivamente si passa all'apertura del socket (con `BACKLOG` definito dal protocollo a 16) ed al processo di bind. Questo si basa sulla correttezza della porta passata in input e sull'output della funzione di bind, se la variabile globale `errno` viene aggiornata con `EADDRINUSE` vuol dire che la porta passata non è disponibile e ne trova automaticamente un'altra libera, provando per un numero definito dal protocollo di `TENTATIVI` tentativi.

Client e server per funzionare necessitano della stessa porta in input e, qualora questa venga scartata dal server perché non valida o occupata, deve esserci un meccanismo di comunicazione verso il client dell'aggiornamento della porta. Il server si fa carico di avviare un thread che indipendentemente dalla validità della porta comunica la porta su cui il server è correntemente in ascolto, ogni qualvolta si riceve un pacchetto `DISCOVER`.

Così facendo si risolve il problema della porta non valida e il problema dell'eventuale disallineamento delle porte passate come input.

Questo meccanismo è definito dalla funzione del thread `udp_discovery_port` e come dice il nome funziona secondo il meccanismo **UDP**: ogni client che richiede la porta la riceve incondizionatamente.

Gestione dei segnali

Durante lo startup e dopo l'apertura dei socket vi è la parte di **gestione delle varie segnalazioni**. Si è ritenuto opportuno gestire le segnalazioni dei segnali di `SIGINT`, `SIGTERM`, `SIGPIPE`, `SIGCHLD`, `SIGALRM`, gestione implementata mediante utilizzo di `sigaction()` e `signal()` (utilizzata come fallback se la `sigaction` non dovesse funzionare).

- `SIGINT/SIGTERM`: una volta ricevute queste segnalazioni viene avviata una funzione di gestione (`chiusura()`) che modifica il valore di una variabile volatile `sig_atomic_t` che permette la **chiusura della lobby** (per effetto cascata) e avvia la chiusura del server. Qualora la `sigaction` restituisca -1 con un `goto` si va alla label `exit` che avvia in automatico lo **shutdown controllato del server**.
- `SIGALRM`: la necessità di controllare questo segnale nasce dal voler assegnare alla lobby un timeout di apertura che una volta scaduto chiude la ricezione di nuove connessioni. Il gestore è `timeout_lobby` che aggiorna la variabile `timeout` (anch'essa volatile `sig_atomic_t`) a 0.
- `SIGPIPE`: gestire questo segnale è d'obbligo ogni qual volta si lavora con canali di comunicazione, quali pipe o in questo caso socket. Generalmente questa segnalazione farebbe terminare il programma, qui si è scelto di ignorarla esplicitamente così che le funzioni di lettura e scrittura sui socket ritornino -1 e il programma possa avviare la sua routine di chiusura controllata.
- `SIGCHLD`: permette di recuperare il valore di ritorno dopo la terminazione del bot, con il `wait(NULL)` lo scarto. Il gestore per questo segnale è `gestore_bot`.

I gestori come si può vedere sono minimali: si occupano di aggiornare una variabile o di fare una `wait()`, tutte procedure che sono **signal-safe**.

Discovery UDP

Come detto precedentemente, vi è un thread dedicato a servire le richieste di DISCOVER da parte dei client e indirettamente comunicare la nuova porta su cui il server è in ascolto. Questo thread viene creato a priori, **indipendentemente dalla validità della porta in input**. Si crea un socket UDP che rimane in attesa della ricezione del messaggio DISCOVER **per tutto il tempo in cui la lobby è aperta**. Una volta scaduto il tempo, il thread va in chiusura e il valore di ritorno scartato (`pthread_detach` nel caso di avvenuta creazione). La funzione del thread è `udp_discovery_port`.

La Lobby

La lobby vive dentro un ciclo `while` con durata che dipende dal valore `TIMEOUT_LOBBY` passato alla funzione `alarm()` definito nel protocollo e dall'avvenuta ricezione o meno di segnalazioni. Nella lobby, il server accetta nuove connessioni in arrivo sul socket. Durante lo sviluppo, specialmente durante la fase di implementazione delle strutture per la sincronizzazione, è emerso un vincolo strutturale: il numero massimo di player. La lobby deve attenersi strettamente a questo vincolo: qualora il nuovo utente che si collega trovasse la lobby piena, il server gli chiude la connessione e chiude la lobby.

Se invece il numero massimo di player è rispettato, si procede con l'allocazione della struttura dati relativa alle informazioni del player connesso, quale socket, indirizzo, il suo `sem_id` e la creazione del thread di gestione dedicato. Questa struttura è necessaria per poter comunicare al thread (tramite `pthread_create()`) più informazioni oltre la singola variabile. La struttura dati in questione è `client_arg`: struct temporanea che contiene le informazioni transitorie che verranno passate poi al thread di gestione.

Una volta avviato, il thread leggerà tali informazioni e le salverà nella struttura dati di gestione dei player connessi `player`, inizializzata come lista a puntatori (poiché a priori non si sa quanti utenti si collegheranno, non può essere inizializzata come array: questo comporta tempi di accesso comunque dipendenti dalla lunghezza perdendo il vantaggio dell'accesso in $O(1)$ al k-esimo elemento). Per tenere traccia di quanti thread attivi ci sono e permettere la sincronizzazione, prima della creazione del thread viene incrementata una variabile globale definita nella `struct gstate: active_threads`, qualora la `pthread_create()` fallisca viene nuovamente decrementata.

Sincronizzazione tra i player

Sincronizzare i player vuol dire garantire il rispettarsi dei turni di gioco. Per poter garantire un corretto ed equilibrato gameplay sono state create strutture dati di sincronizzazione IPC: i semafori. Come accennato precedentemente ci sono **due insiemi di semafori tra loro distinti**: `sem1` serve per “passare il gettone tra client” mentre `sem2` dà il via libera alla partita.

La logica di gestione di questi semafori è molto semplice:

- `sem1`: implementato secondo una logica di gestione di tipo *round-robin* passando il “gettone” in modo circolare seguendo la logica modulare $(i+1) \% n$. Una gestione dei turni puramente basata su logica modulare sarebbe stata sufficiente qualora il numero di client connessi non fosse variato nel corso del gameplay; poiché a priori non si sa quale giocatore si disconnette o viene eliminato, la logica modulare viene adottata solo per la ricerca del primo player ancora in vita. Nella logica di gioco ci sono frammenti di codice che si occupano della gestione del prossimo turno: qualora il player colpisca una nave **non** deve passare il turno al prossimo player, ma mantenerlo fin quando non compirà un `MISS`. Nel caso di `ELIMINATED` o `MISS`, nella label `next_turn` si ricerca il prossimo player vivo, recuperando il suo `sem_id`, gli si passa il “gettone”, ricerca protetta da mutex poiché si accede ad informazioni dei player in modo concorrente.
- `sem2`: questo semaforo funge da blocco all'attività dei player. Si tratta di un semaforo con ruolo quello di contatore, sul quale verranno fatte le post. Inizializzato allo startup a 0, con `sem_op = -slot` dove `slot` è il numero di connessioni effettivamente accettate durante la lobby, sbloccherà il main non appena verranno effettuate `slot` post. La necessità di dimensionare sul numero di connessioni accettate e non sul numero di player che hanno completato l'handshake, è sorta durante la fase di test con un client che presentava alta latenza. Si è visto che una connessione potrebbe risultare accettata, con il proprio thread relativo attivato, prima che completi l'handshake. Usando come dimensione il numero di giocatori registrati, il main aspettava meno post di quante ne sarebbero arrivate, avviando così la partita in anticipo. Vista la possibilità che un thread termini anche prima di aver completato l'handshake, la post sul semaforo è fatta anche in fase di uscita: con un flag `posted` si controlla l'avvenuta `post` su `sem2`.

Una nota importante va fatta per `sem1`. Questo semaforo come detto si occupa di regolare i turni tra i client. Poiché va necessariamente inizializzato prima della lobby in quanto per costruzione l'accettazione di un client comporta la creazione di un thread, è sorto un problema di dimensionamento.

Dovendo avere una dimensione almeno pari ($\geq$) al numero di client presenti al momento di chiusura lobby e dovendo necessariamente essere creato *prima* della lobby, è stato necessario introdurre un numero massimo di player (protocollo) che definisce la dimensione a priori. Anche se è una sovrastima è stata una scelta obbligata, causa le dimensioni fisse delle strutture semaforiche.

Client_thread

Ad ogni connessione da parte di un nuovo client, viene delegata la gestione ad un thread apposito: `client_thread` che si occupa di: assegnare l'id al momento della connessione; di registrare l'username scelto dal player; di creare tutte le strutture dati necessarie e popolarle con le varie informazioni. Come messaggio di ACK, il server risponde alla richiesta di `JOIN` con un `WELCOME` **solo quando la connessione è stata stabilita**, secondo quindi un meccanismo di handshake basato su connessione TCP. Poiché si è scelto di poter far selezionare la modalità di gioco desiderata, oltre a ricevere l'username, viene letta anche la modalità di gioco scelta. Se il client connesso è il primo in lobby può impostare la modalità di gioco **atomicamente** (assegnazione protetta da mutex di `gstate`), altrimenti si deve attenere alla modalità correntemente selezionata dalla prima connessione effettuata. Di default la modalità di gioco della lobby è quella richiesta da specifica: `TcT`.

Le modalità di gioco sono due: **1v1** e **TcT**. La modalità **TcT non impone vincoli stringenti** a differenza della **1v1**. Poiché in questa modalità si gioca rigorosamente in due giocatori, il server rifiuta ogni altra connessione qualora vi siano già connessi due player. Nonostante il raggiungimento del numero di player necessari, la lobby non chiude ma rimane aperta così, qualora uno dei due client connessi si disconnetta, la lobby sarebbe pronta ad accettare un nuovo player per colmare il buco creato. Se allo scadere del timer della lobby vi è ancora una sola connessione accettata, il server non chiuderà la connessione del player, ma lancerà il bot (rifiutando qualsiasi altro tentativo di connessione da altri utenti nella finestra che si crea con il blocco della lobby e la chiusura del socket). Tramite una `fork()` e poi un `execl()` nel figlio, si può eseguire il Bot che si conatterà a tutti gli effetti come nuovo client.

Una volta stabilita la connessione, il server accetta un nuovo pacchetto di dati contenente la flotta pre-validata dal client, qui viene effettuato un secondo controllo e se accettata viene inserito definitivamente in partita, altrimenti gli verrà chiusa la connessione.

Finché la partita risulta in corso e finché il client risulti in vita, vengono effettuati i controlli sulla ricezione delle mosse quale validazione della mossa e controllo di `HIT` o `MISS`.

Logica di gioco

Per implementare il corretto funzionamento del gioco si è seguita la classica logica già adottata dal vero gioco: qualora un attacco vada a segno (`HIT`) il player mantiene il turno; qualora si mancasse il bersaglio (`MISS`) il turno è ceduto al player successivo. Questa logica risulta quindi rispecchiata in un meccanismo a turni circolare che nel caso del gioco `1v1` necessiterebbe a priori di passare il turno al prossimo player senza vari altri controlli, nella variante multiplayer questo approccio va rivisto. Quando un player riceve come riscontro un `MISS` dal server, il turno non passa al giocatore che si trova subito dopo di lui: a priori non si può stabilire se il player `i+1` sia vivo o meno, vanno quindi controllate informazioni relative

al client immediatamente successivo e agire di conseguenza. Per prevenire *deadlock* dovuti al rilascio del turno ad un client terminato o disconnesso, sono state introdotte delle funzioni ausiliarie esclusive del server (`trova_giocatore` per identificare il player target -tramite un flag si specifica se la ricerca è basata su `id` o su `sem_id`; `broadcast` che comunica a tutti l'esito della mossa) che permettono di identificare tramite `id` o `sem_id` il primo player che risulti ancora in gioco.

Il flusso del gameplay è quindi questo:

⇒ si aspetta il proprio turno ⇒ si recuperano le informazioni relative al player ⇒ viene comunicata al player corrente la lista dei giocatori correntemente in vita (`id` e `USERNAME`) così da poter scegliere il bersaglio su cui fare fuoco.

Successivamente viene comunicato al client tramite un pacchetto di informazioni che è il proprio turno ed impostato un tempo di inattività pari ad un valore definito dal protocollo (`AFK_TIMEOUT`). Tale timeout è realizzato mediante l'opzione `SO_RCVTIMEO` su socket installata mediante `setsockopt()`: quando scade il timeout, la `recv` fallirà impostando `errno = EAGAIN`, permettendo così di distinguere una disconnessione per inattività o di altra natura (che lascia `errno` a zero). Viene poi ricevuta la mossa dal player (con posizioni già verificate dal client se nei limiti) e verificato poi l'eventuale `HIT` o `MISS`. A seconda dell'esito viene quindi applicato il meccanismo di turnazione definito precedentemente, effettuati dei controlli per l'eventuale fine partita causa vittoria o eliminazione e successivamente eventuale disconnessione.

End Game e chiusura

A fine partita - ovvero quando non ci sono più client perché o eliminati o per disconnessione - si avvia una **routine di chiusura definita sia nel main che nei thread**. Qualora si venga eliminati o si vinca il gioco, il thread va in chiusura de-allocando la struttura dati relativa al proprio player e successivamente chiude il socket del player connesso. Il main invece rimane in uno stato di `sleep()` indefinito: ogni 1s si risveglia, controlla quanti thread attivi ci sono e qualora siano 0 oppure si riceve la segnalazione `SIGINT` o `SIGTERM`, avvia la routine di chiusura che: chiude i socket in ascolto se (nel caso di segnalazione prima della chiusura lobby) ancora aperti; rimuove le strutture `IPC` relative ai semafori; aspetta che tutti i thread correntemente attivi terminino entro un `TIMEOUT_SHUTDOWN` definito dal protocollo e rilascia la memoria della `struct player` relativa ai player connessi.

Protocollo di comunicazione

Per poter far comunicare correttamente client e server in rete è sorta la necessità di creare un protocollo di comunicazione, composto da varie struct che lavorano insieme.

Tra le strutture principali in questo progetto quindi troviamo:

```
typedef struct azioni{                                // definisce pacchetti scambiati client-server
    game_info type;                                    // specifico il tipo di azione (enum)
    uint8_t player_id;                                // id del player che compie un'azione
    uint8_t target_id;                                // id del player da colpire
    uint8_t x,y;                                       // posizione nave da colpire
    mode gamemode;                                    // comunica la modalità di gioco insieme a MODE
    char username[USERNAME];                           // comunica l'username al momento del JOIN
}
```



```

    uint8_t affondata;           // comunica l'indice della nave affondata (HIT)
} azioni;

typedef struct posizionamento{ // definisce la nave durante il posizionamento
    uint8_t index;              // indice in ship_type
    uint8_t x,y;                // coordinate del posizionamento nave
    char orientation;           // orientamento della nave (N,S,E,O)
}posizionamento;

```

La struct `azioni` insieme all'enum `game_info` permette di comunicare informazioni tramite tipo al client quali HIT MISS JOIN WELCOME INFO ecc... le altre invece si occupano di trasportare in rete informazioni relative al tipo di mossa, nave e target.

Per ridurre l'*overhead* del payload in rete si è optato per l'utilizzo di `uint8_t` per i valori relativi alle coordinate e agli id. Interi a 8 bit *endianness free*, riducono i vari controlli e trascrizioni in fase di ricezione (`htonl` o `ntohl` si applicano solo a `type` e `gamemode`, interi a 4 byte). Tra i vari dati contenuti in `azioni`, vi è `uint8_t affondata`: questa variabile comunica l'indice della nave che è stata affondata; qualora non sia ancora stata affondata nessuna nave occorre comunicare un valore di riferimento: in protocollo definito sotto il nome `NESSUNA_NAVE`, vi è il valore `SHIP_NUMBER`, che non corrisponde a nessun indice valido in `ship_type` e, poiché è un array, ha indici fino a `SHIP_NUMBER-1`.

Per poter ridurre al massimo tale *overhead* e per garantire l'ordine dei byte in rete (e per garantire la portabilità) le strutture dati più importanti sono racchiuse sotto la direttiva del preprocessore `#pragma pack()`, che allinea i dati al byte.

Bot come nuovo player

Poiché si è scelto di chiedere all'utente quale modalità di gioco volesse usare, è sorto il dubbio su cosa fare qualora la lobby sia formata da un solo client e sia stata scelta la modalità 1v1. Per non dover scollegare forzatamente il client per assenza di player si è scelto di realizzare un bot. Tale Bot viene lanciato dal server soltanto qualora vi sia una sola connessione accettata, indipendentemente dalla modalità di gioco. Lanciato esclusivamente dal processo del server, è scritto per standard POSIX, è una variante (priva quindi dei vari controlli degli input) del client.

Client

A differenza del server, il codice del client è pensato per essere eseguito su macchine di famiglia diversa: il codice è quindi cross-platform.

Il codice sorgente è comprensivo di direttive al preprocessore per la compilazione per standard POSIX e standard WINAPI.

Le differenze implementative necessarie per il cross-platform verranno poi elencate e discusse in una sezione apposita. Il vantaggio di non dover riscrivere gran parte del codice del client è stato reso possibile grazie alla creazione dei file `utils.h` e `utils.c` contenenti rispettivamente le firme e le definizioni di funzioni condivise tra client e server. Le funzioni qui presenti sono proprio quelle dedite alla comunicazione su socket. L'inclusione di questi

due nuovi file nel progetto ha reso lo sviluppo del codice tra client e server molto più semplice ed ha garantito la facile portabilità tra ambienti WINAPI e POSIX.

Startup del client

In maniera analoga a quella del server, lo startup consiste nel creare l'eseguibile tramite comando del Makefile `make client` (per standard POSIX) e `make wclient` (per standard WINAPI). Successivamente va lanciato seguendo lo schema `./battaglia_client <ip> <port>` per POSIX oppure per WINAPI `./battaglia_client.exe <ip> <port>` passando la stessa porta su cui è in ascolto il server. Qualora il client risieda sulla stessa macchina che ospita l'eseguibile del server, si deve passare 127.0.0.1 come ip, altrimenti su macchine differenti va passato l'ip esplicito come parametro. Se le macchine che ospitano server e client si trovano sulla **stessa rete locale**, in alternativa si può attivare l'*auto mode*: nessun input passato al momento dello startup.

Così facendo si avvia il client che inizierà con l'installazione degli handler per le segnalazioni per poi effettuare i controlli preliminari sulla plausibilità dei dati forniti e infine aprire i socket.

Differenze tra POSIX e WINAPI

La distinzione tra i due standard viene fatta sfruttando la programmazione condizionale (`#ifdef _WIN32`) e le differenze caratterizzano:

- *i socket*: poiché standard WINAPI, l'apertura di un socket restituisce un tipo `SOCKET` anziché un intero, è sorta la necessità di adattare tutte le funzioni che (secondo standard POSIX) ricevevano file descriptor per il socket in modo tale che ricevessero (o restituisce) un `SOCKET`. Per i socket si è usato `WSAStartup` per inizializzare la libreria, `socket()` per aprirlo, mentre per chiudere `closesocket()`. Per agevolare la chiusura dei socket e snellire il codice, si è delegato il compito ad una funzione ausiliaria `chiudi_socket()` presente in versione winapi e posix.
- *segnalazioni*: non essendoci i segnali come in POSIX (non si poteva usare la `sigaction()`), in standard WINAPI gli eventi di controllo vengono gestiti tramite handler registrato da `SetConsoleCtrlHandler()`.
- *gestione dei codici ANSI*: come detto, essendo che windows non supporta nativamente i codici colore ANSI ma vanno attivati, si è usata la funzione `SetConsoleMode()` con l'attributo `ENABLE_VIRTUAL_TERMINAL_PROCESSING`
- *timeout per i socket*: invece di usare la struct `timeval`, si è usata `SO_RCVTIMEO` (DWORD)
- *funzioni di lettura e scrittura su/da socket*: le funzioni di utility sono state riscritte (o adattate) per standard WINAPI.

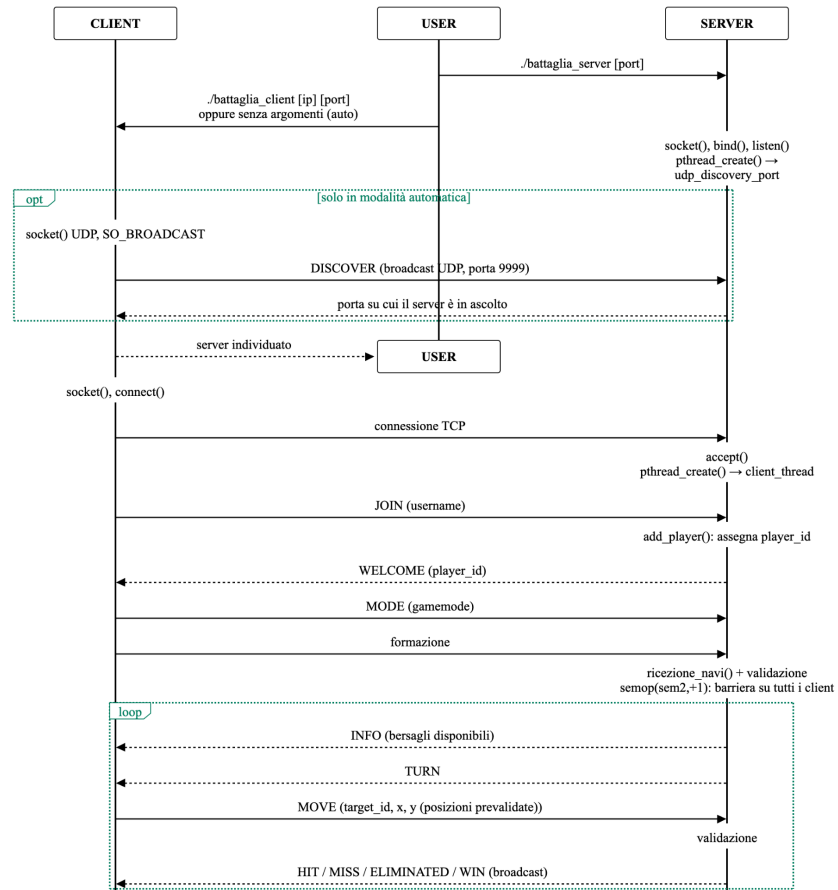
Connessione al server

La connessione al server viene effettuata secondo due modalità: **modalità automatica** (che trova la porta di ascolto del server in automatico) e **modalità manuale** (che si basa sulla porta in ingresso). La modalità automatica viene avviata solo quando non c'è input di dati al momento dello startup del client; inoltre si ripresenta come meccanismo di fallback qualora

ip e porta passati come parametri non siano validi. Tale meccanismo quindi invoca una funzione per la ricerca del server: qui viene aperto un socket in UDP che manda una richiesta di `DISCOVER` in broadcast sulla rete, usando l'indirizzo di broadcast `x.x.x.255:9999`, dove la porta di broadcast è definita comune nel protocollo tra server e client, `DISCOVERY_PORT`. Una volta inoltrata la richiesta di discovery, il server la riceve e risponde comunicando la porta dove è correntemente in ascolto, successivamente viene registrata dal client che effettuerà una `connect()` tramite la funzione ausiliaria `connetti()` sulla porta ricevuta. Nel payload non è necessario inviare anche l'ip in quanto è un'informazione che può essere recuperata con la struct `sockaddr_in`.

Successivamente viene effettuato l'handshake con il server: viene mandato un payload dati contenente il tipo `JOIN` e il server lo accetta rispondendo con `WELCOME`.

Questo meccanismo funziona in maniera analoga tra standard diversi, questo è stato possibile mediante la dichiarazione di una funzione ausiliaria `connetti()` al cui interno si fa la distinzione tramite programmazione condizionata.



Ricezione informazioni utente e inizializzazione griglia

Una volta stabilita la connessione con il server, viene chiesto al player l'username tramite la funzione ausiliaria `getUsername()`. Successivamente il client lo comunicherà al server insieme al pacchetto dati di `JOIN`. Una volta ufficializzata la connessione (meccanismo di handshake `JOIN-WELCOME`) verrà chiesta all'utente la modalità di gioco desiderata, inviata poi successivamente tramite pacchetto dati di tipo `MODE`.

Si è deciso di attuare un meccanismo di **salvataggio connessioni** - quando un utente si collega per la prima volta al server, la funzione controlla se nella directory corrente è presente un file `_user`. Se il controllo va a buon fine la funzione legge il nome contenuto nel file e procede con il flusso d'esecuzione. Qualora il file non sia presente vuol dire che il client si sta collegando per la prima volta al server, quindi chiede da terminale l'username e lo salva nel file: qualora la creazione del file non vada a buon fine il player giocherà in modalità ospite.

Ricevute queste informazioni il client le inoltra al server e attende il piazzamento delle navi da parte del player. Per fare ciò ci sono funzioni dedite alla ricezione delle posizioni e

all'aggiornamento in tempo reale della griglia (`piazzamento_navi()` + `gui.h`), oltre a queste vi è una funzione di prevalidazione prima dell'invio delle navi (`validazione()`).

Prima della ricezione della flotta, si chiama la funzione `init_board()` che inizializza la griglia condivisa tra client e GUI, facendo la distinzione tra griglia nemica e propria. Fatto ciò si inizia con la ricezione della formazione.

Una volta completata la ricezione delle navi dall'utente, validata la flotta, tramite la funzione `invio_navi()` (e con la sua gemella in server `ricezione_navi()`) si invia la flotta al server che effettuerà poi una seconda validazione.

Tutto il lavoro relativo alla parte grafica e all'interazione con l'utente è stato delegato ad una libreria `gui.h` che contiene tutte le funzioni della GUI.

Gameplay

A differenza del server che per ogni client doveva delegare il lavoro ad un thread apposito, qui è presente solo il main thread che riceve le informazioni e inoltra le mosse tramite socket in un while che esegue fin quando non si riceve una segnalazione per disconnessione oppure un game over. All'interno è presente uno switch per distinguere tra i vari casi. Il server comunica le varie informazioni specificando il proprio tipo (dal protocollo) e il client agisce di conseguenza. Quando il server lascia al client il turno, viene chiamata una funzione di GUI che si occupa di ricevere la mossa dall'utente e il target selezionato, all'interno vengono effettuati i vari controlli sulla plausibilità della cella selezionata. Se i dati inseriti sono corretti viene poi popolato un pacchetto di informazioni che restituito al main del client viene comunicato al server. Nei vari casi non c'è una vera distinzione tra `HIT` e `MISS` poiché cambia solo il tipo di notifica, quindi il `case HIT` ricade nel `case MISS`: se viene colpito un bersaglio (controllando `pck.type`) viene aggiornata la griglia con un 'X' altrimenti un 'O' ed a seconda del tipo di risultato si stampa a schermo un messaggio diverso, distinguendo il caso in cui il colpo abbia affondato o meno una nave. Come detto precedentemente l'indice della nave viene ricavato dalla variabile `affondata` in `azioni`.

Qualora al client venga decretata la vittoria, si avvia la procedura di chiusura: il server manda in broadcast un messaggio di tipo `WIN` e ogni client stamperà il messaggio di fine partita e terminerà. Poiché i messaggi che inoltra il server sono in broadcast quindi verso tutti i client connessi, si può fare da spettatore anche dopo l'eliminazione: tramite GUI si possono seguire gli sviluppi della partita (**modalità spettatore**).

Interfaccia Utente

Entrambe le release (WINAPI e POSIX) usano un'interfaccia grafica basata su riga di comando (CLI). Il compito di stampare a schermo gli aggiornamenti della partita viene delegato ad una libreria grafica, `gui.h` che, come detto, contiene tutte le funzioni per stampare a schermo le varie informazioni. L'interfaccia fa uso di codici ANSI per colorare i vari messaggi di errore e le varie celle colpite, distinguendo tra quelle colpite o meno.

Inizialmente per ripulire il terminale da messaggi precedenti veniva utilizzata la funzione `system("clear")`. Tale funzione è perfetta per ambienti POSIX ma in ottica di portabilità non poteva essere usata in ambienti WINAPI. Per risolvere questa discrepanza si è scelto di utilizzare il codice ANSI `\033[H\033[J` che permette di ripulire il terminale ed è "cross-platform". Nonostante questa scelta, le codifiche ANSI sono abilitate nativamente su

console di standard POSIX, per standard WINAPI no e vanno abilitate. Nel codice del client sotto i define per `_WIN32` e all'inizio del codice, vengono abilitati esplicitamente mediante l'attributo `ENABLE_VIRTUAL_TERMINAL_PROCESSING` di `SetConsoleMode()`.

Chiusura

Il procedimento di chiusura anche qui è centralizzato: il main quando riceve segnalazioni oppure ci sono errori dovuti a installazioni di componenti avvia la chiusura. Poiché il codice è cross-platform, si è introdotta una funzione ausiliaria `chiudi_socket()` che permette di chiudere il socket sia nel caso WINAPI che POSIX. Il processo di chiusura è accompagnato da notifiche tramite GUI.

Bot

Come detto in precedenza, il Bot è una variante del client priva di input forniti dall'utente. Ci si limiterà quindi alla descrizione delle funzioni non presenti in client, ma di cui il Bot necessita per il suo corretto funzionamento. Per la compilazione si può digitare `make bot`, in alternativa digitare soltanto `make server` e l'eseguibile viene generato in automatico.

Startup del Bot

Il Bot viene lanciato esclusivamente dal server **solo quando** la **lobby** è formata da **una sola connessione accettata**. All'interno del codice del server quindi, subito dopo la chiusura lobby, vi è un controllo: se è stata accettata una sola connessione, allora tramite `fork()+execl()` ne viene lanciato l'eseguibile, gestito come se fosse a tutti gli effetti un player reale (connessione tramite socket, scelta di id, username hardcoded a BOT). Poiché viene avviato esclusivamente dal server con porta e ip già validati, il Bot si limita a un *controllo minimale* sulla presenza di argomenti.

Piazzamento delle navi

La funzione che si occupa del piazzamento delle navi (`piazzamento_navi()`) è rimasta pressoché identica a quella presente nel client, ad eccezione della scelta delle coordinate iniziali di ogni nave, che sono generate casualmente.

Selezione della mossa

La selezione della mossa avviene tramite la funzione `selezione_prossima_mossa()`. Quest'ultima si serve di un flag (`inseguimento`) per decidere come impostare la mossa successiva:

- `inseguimento` viene impostato a `true` nel momento in cui un colpo va a segno.
 - qualora il colpo affondi una nave (variabile `affondata`), si interrompe l'inseguimento e si resettano i vari flag.
- se `inseguimento = false` il Bot cercherà una casella disponibile tramite la funzione `ricerca()` in modo casuale. Questa funzione seleziona una nuova cella non ancora utilizzata andando a consultare una matrice

`tentativi[GRID_SIZE][GRID_SIZE]`, azzerata qualora tutte le celle venissero tentate.

- se `inseguimento = true` e `direzione_colpi = -1`, ovvero non è ancora nota una direzione lungo cui sparare, il Bot cercherà una cella adiacente a quella del colpo iniziale andato a segno, per trovare l'orientazione giusta della nave a cui sta sparando.
- se `inseguimento = true` e `direzione_colpi` è fissata, il Bot continuerà a sparare lungo la direzione scelta.

La funzione `selezione_prossima_mossa()` prevede la presenza di un flag aggiuntivo (`riprovato_verso_opposto`). Il flag serve per capire se è possibile invertire la direzione con cui il Bot spara, qualora il colpo vada a vuoto oppure si raggiunga il bordo della griglia. In caso negativo il Bot sceglierà la prossima mossa tramite `ricerca()`.

Una volta provato il verso opposto e trovata la prima cella non disponibile, `inseguimento` viene impostato a `false` e la scelta delle mosse successive avverrà tramite `ricerca()`.

Validazione scelta delle celle

La validazione della scelta delle celle avviene tramite la funzione `cella_papabile()`, la quale restituisce `true` nel momento in cui è possibile sparare nella cella scelta.

Codice sorgente ed esecuzione

Di seguito vengono riportati tutti i sorgenti del progetto e le istruzioni per la compilazione.

Manuale d'uso

Struttura delle directory e istruzioni più dettagliate presenti nel README della [repository](#).

Prerequisiti

Per il corretto funzionamento, in ambiente POSIX sono richiesti il compilatore gcc e make. Il server e il Bot sono compilabili **solo** per standard POSIX. Il client è cross-platform; per la release WINAPI, test e sviluppi sono stati condotti su shell MSYS2 (UCRT64) dopo aver installato la toolchain con il comando `pacman -S mingw-w64-ucrt-x86_64-gcc make`.

La compilazione

Il processo di compilazione è automatizzato tramite Makefile presente nella directory principale del progetto. Di seguito i comandi per la compilazione:

<code>make all</code>	compila secondo lo standard POSIX server, client e bot.
<code>make wclient</code>	compila secondo lo standard WINAPI il client.
<code>make clean</code>	Rimuove tutti gli eseguibili.
<code>make help</code>	Mostra il riepilogo dei comandi disponibili.

Per compilare singolarmente i vari moduli ci sono i comandi: `make server`, `make client`, `make bot`.

Il target `server`, così come `all`, genera anche l'eseguibile del Bot in quanto, per come strutturato, il server si aspetta di trovarlo al momento dell' `execl()`. Il Bot non va mai eseguito manualmente.

Esecuzione

Server

Per poter eseguire il server, digitare `./battaglia_server <port>` con $5000 \leq \text{port} \leq 65535$. Qualora la porta risulti invalida o occupata, si attivano i meccanismi di fallback. Eseguire dalla directory principale del progetto.

Client

Per eseguire il client digitare `./pname <ip> <port>` con ip quello del server (in locale 127.0.0.1) oppure digitare `./pname` per eseguire secondo il discovery automatico. Sostituire `pname` con `battaglia_client` (POSIX) o `battaglia_client.exe` (WINAPI).

Note operative: la modalità automatica è utilizzabile solo quando server e client si trovano sulla stessa rete locale: il meccanismo si basa sull'utilizzo di messaggi broadcast su porta UDP 9999 che deve risultare sempre raggiungibile. Questo è stato il motivo per cui si è utilizzata la VPN ZeroTier. Su reti differenti o reti dove la modalità broadcast non funziona, si deve utilizzare la modalità manuale, in alternativa si può collegare la VPN.

Al primo avvio del client, viene creato un file `_user` che memorizza l'username scelto, così da non chiederlo ad ogni connessione ma solo alla prima. Qualora si volesse reimpostare, si può rimuovere digitando `make clean`.

server.c

```
/*
    SERVER POSIX RELEASE
*/
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>

#include "../protocollo/protocollo.h"
#include "../utils/utils.h"
#include <unistd.h>
#include <errno.h>
#include <signal.h>
#include <pthread.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/mman.h>
#include <sys/sem.h>
#include <semaphore.h>
#include <fcntl.h>
```

```

#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <arpa/inet.h>
#include <time.h>
#include <sys/wait.h>

typedef struct players player;

volatile sig_atomic_t timeout = 1, lobby_aperta = 1, shutdown_flag = 0, sig_ricevuta = 0;
int sem1 = -1, sem2 = -1, slot = 0; /*slot: dimensiona sem1 e sem2*/
unsigned int port;
/*
    sem1-> serve per i turni
    sem2-> serve per dare il via libera a tutti dopo che anche l'ultimo client connesso ha
    finito di disporre le navi
    n-> mi serve per il numero totali di gettoni che andranno nei semafori
*/

void *udp_discovery_port(void *args);
void timeout_lobby(int sig);
void chiusura (int sig);
void gestore_bot (int sig);
void *client_thread (void *args);
void *add_player(int fd, char *username, int sem_id);
void remove_player(int id, bool flag);
player *trova_giocatore(int valore, bool flag); // serve per trovare un giocatore in base al
suo id quando si vuole fare una
// mossa contro di lui, così da poter aggiornare la sua
griglia e il numero di navi rimaste
// si può cercare tramite id (flag = false) oppure sem_id
(flag = true)
bool validazione_formazione(char board[GRID_SIZE][GRID_SIZE], int x, int y, char orientazione,
uint8_t dimensione_nave, int indice); // serve per il check della formazione in entrata,
INTEGRITY CHECK
int ricezione_navi(int fd, player *me); // serve per ricevere la formazione che manda il
client
void broadcast(azioni *esito); // serve nel client_thread per fare le comunicazioni a tutti
gli utenti

//serve perchè ho bisogno di passare più informazioni al thread, uso quindi una struttura
typedef struct{ // describe il client
    int client_fd;
    struct sockaddr_in client_addr;
    int sem_id; // valore transitorio che poi viene passato come argomento al thread client
    che lo inserisce nella struct privata player
}client_arg;

typedef struct players{
    int id;
    char username[USERNAME];
    int socket;
    char griglia[GRID_SIZE][GRID_SIZE]; // griglia giocatore per validare le mosse
    int alive;
    int navi_rimaste;
    int sem_id;
    struct players *next;
}player;

```



```

typedef struct{
    player *head;
    int count; // giocatori effettivamente registrati
    int next_id; // serve per il prossimo id
    int fine;
    int active_threads; // serve per tenere traccia di quanti thread client sono attivi, così
da poterli chiudere tutti in caso di poweroff
    mode modalita;
    int mode_scelta;
    pthread_mutex_t lock; // serve per proteggere questi valori, senza che lo dichiaro globale
lo metto qui dentro
} gstate;

static gstate stato = {
    NULL,
    0, // quanti giocatori sono connessi
    0, // quanti utenti sono già connessi
    0, // inizializzazione flag per la fine
    0, // inizialmente nessun thread attivo
    MODE_DEFAULT, // modalità di gioco di default
    0, // se 0 ancora non scelta, appena diventa 1 è scelta
    PTHREAD_MUTEX_INITIALIZER
};

int main(int argc, char **argv){

    if(argc<2){
        fprintf(stderr,"Sintassi corretta: %s <number>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    srand((unsigned int)time(NULL));

    int porta = atoi(argv[1]);
    if(porta< 5000 || porta >65535){
        printf("Devi inserire un numero di porta valido nel range 5000-65535!\n");
        printf("Avvio la ricerca automatica della porta...\n");
        port = 5000 + (rand() % (65535 - 5000));
    } else port = (unsigned int) porta;

    printf("\n          BATTAGLIA NAVALE - SERVER v1.0          \n");
    printf("[*] Startup del server in corso...\n");
    fflush(stdout);

    int llisten = socket(AF_INET, SOCK_STREAM, 0);
    if (llisten < 0) {
        perror("Errore nel socket");
        exit(EXIT_FAILURE);
    }

    int opt = 1, ret;
    ret = setsockopt(llisten, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
    if(ret == -1){
        perror("Errore nella setsockopt");
        goto exit;
    }
}

```

```

struct sockaddr_in server_addr;
memset(&server_addr, 0, sizeof(server_addr));

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(port);
server_addr.sin_addr.s_addr = htonl(INADDR_ANY); // in ascolto su tutte le interfacce

int bound = 0; // appena ho successo mi fa uscire dal for
for (int attempt = 0; attempt < TENTATIVI && !bound; attempt++) {
    server_addr.sin_port = htons(port);

    if (bind(llisten, (struct sockaddr *)&server_addr, sizeof(server_addr)) == 0) {
        bound = 1;
    } else if (errno == EADDRINUSE) {
        printf("[*] Porta %d occupata, provo una porta casuale...\n", port);
        port = 5000 + (rand() % (65535 - 5000)); /* range non privilegiato */
    } else {
        perror("bind() fallita");
        goto exit;
    }
}

if (!bound) {
    fprintf(stderr, "Impossibile trovare una porta libera dopo il numero massimo di
tentativi concesso");
    goto exit;
}

// dimensione del backlog -> dimensione della coda di attesa (pending connection non
ancora accettate), definita nel protocollo
if (listen(llisten, BACKLOG) < 0) {
    perror("listen() fallita");
    goto exit;
}

printf("\n");
printf("=====\n");
printf("          SERVER AVVIATO CON SUCCESSO          \n");
printf("=====\n");
printf(" Progetto Sistemi Operativi - A.A. 2025/2026\n");
printf(" Lorenzo Tarantino - Leonardo Rocco Cicalini\n");
printf("=====\n");

printf(" Indirizzo IP : 0.0.0.0 (INADDR_ANY)\n");
printf(" Porta          : %u\n", port);
printf(" Backlog        : %d (Coda massima client)\n", BACKLOG);
printf(" Per chiudere : CTRL + C\n");

printf("=====\n");
printf("          Connessione al server          \n");
printf("=====\n");

printf(" Da questa macchina : ./battaglia_client 127.0.0.1 %u\n", port);
printf(" Da macchine diverse: ./battaglia_client <ip_server> %u\n", port);
printf(" In rete locale      : ./battaglia_client (auto mode)\n");

printf("=====\n");
printf("[*] In attesa di nuove connessioni...\n");

fflush(stdout);

```

```

struct sigaction sa;
sa.sa_flags = 0;

if (sigemptyset(&sa.sa_mask) == -1){
    perror("Errore nello svuotare la maschera delle segnalazioni");
    goto exit;
}

sa.sa_handler = SIG_IGN;

if(sigaction(SIGPIPE, &sa, NULL) == -1){
    perror("Errore nell'installare la sigaction per SIGPIPE, procedo con l'installazione
della signal");
    if(signal(SIGPIPE, SIG_IGN) == SIG_ERR){
        perror("Errore nell'installazione della signal");
        goto exit;
    }
}

sa.sa_handler = timeout_lobby;

if(sigaction(SIGALRM, &sa, NULL) == -1){
    perror("Errore nell'installare la sigaction per SIGALRM, procedo con l'installazione
della signal");
    if(signal(SIGALRM, timeout_lobby) == SIG_ERR){
        perror("Errore nell'installazione della signal");
        goto exit;
    }
}

sa.sa_handler = chiusura;

if (sigaction(SIGINT, &sa, NULL) == -1) {
    perror("Errore nell'installare la sigaction per SIGINT, procedo con l'installazione
della signal");
    if(signal(SIGINT, chiusura) == SIG_ERR){
        perror("Errore nell'installazione della signal");
        goto exit;
    }
}

if (sigaction(SIGTERM, &sa, NULL) == -1) {
    perror("Errore nell'installare la sigaction per SIGTERM, procedo con l'installazione
della signal");
    if(signal(SIGTERM, chiusura) == SIG_ERR){
        perror("Errore nell'installazione della signal");
        goto exit;
    }
}

sa.sa_handler = gestore_bot;
if(sigaction(SIGCHLD, &sa, NULL) == -1){
    perror("Errore nell'installare la sigaction per SIG_CHILD, procedo con l'installazione
della signal");
    if(signal(SIGCHLD, gestore_bot) == SIG_ERR){
        perror("Errore nell'installazione della signal");
        goto exit;
    }
}
}

```

```

pthread_t udp_thread;

if(pthread_create(&udp_thread, NULL, udp_discovery_port, &port) != 0){
    perror("Errore nella creazione del thread per la discovery UDP");
    goto exit;
} else {
    pthread_detach(udp_thread);
    printf("[*] Thread UDP discovery creato con successo\n");
    fflush(stdout);
}

sem2 = semget(IPC_PRIVATE, 1, IPC_CREAT|0664);
if(sem2 == -1){
    perror("Errore nella creazione del semaforo per il controllo che tutti i player hanno
terminato l'invio della formazione");
    goto exit;
}

semctl(sem2, 0, SETVAL, 0); // inizialmente ho 0 gettoni, poi mi faccio fare le post su
questo semaforo e quando torna a 0 sblocco tutto

sem1 = semget(IPC_PRIVATE, MAX_PLAYER, IPC_CREAT|0664);
if(sem1 == -1){
    perror("Errore nella creazione del semaforo per i turni dei client connessi");
    goto exit;
}

int current_player = 0; // variabile locale al while per il controllo dei client
collegati, senza che uso il mutex
printf("[*] Lobby aperta per %ds\n", TIMEOUT_LOBBY);
alarm(TIMEOUT_LOBBY);

//lobby con timeout
while(timeout && !shutdown_flag){
    struct sockaddr_in client_addr;
    socklen_t addrlen = sizeof(client_addr);

    int client = accept(llisten, (struct sockaddr *)&client_addr, &addrlen); // bloccante
    if(client == -1){
        if(errno == EINTR) continue;
        perror("Errore nell'accept() in ascolto del client");
        continue;
    }

    if(current_player >= MAX_PLAYER){
        close(client);
        alarm(0);
        break;
    }

    client_arg *cargs = malloc(sizeof(client_arg));
    if(cargs == NULL){
        perror("Errore nella malloc");
        close(client);
        continue;
    }

    //mi salvo tutte le informazioni riferite al client connesso nella struttura e invio
    poi al thread di gestione

```

```

    cargs->client_fd = client;
    cargs->client_addr = client_addr;
    cargs->sem_id = current_player;

    pthread_t tid;

    pthread_mutex_lock(&stato.lock);
    stato.active_threads++;
    pthread_mutex_unlock(&stato.lock);

    if(pthread_create(&tid, NULL, client_thread, cargs) != 0){
        perror("Errore nella creazione del thread di gestione associato al client
connesso");
        pthread_mutex_lock(&stato.lock);
        stato.active_threads--;
        pthread_mutex_unlock(&stato.lock);
        close(client);
        free(cargs);
        continue;
    }
    pthread_detach(tid);
    current_player++;
}

lobby_aperta = 0;

if(current_player == 1 && !shutdown_flag){
    printf("[*] Attenzione: un solo client collegato, startup del bot...\n");
    fflush(stdout);
    pid_t pid = fork();
    if(pid == 0){
        char porta[10];
        snprintf(porta, sizeof(porta), "%u", port);
        execl("./battaglia_bot", "battaglia_bot", "127.0.0.1", porta, NULL);
        perror("Errore nello startup del bot");
        exit(1);
    } else if (pid < 0){
        perror("Errore nella fork per l'avvio del bot");
    } else {
        struct sockaddr_in bot;
        socklen_t lbot = sizeof(bot);
        struct timeval tv;
        tv.tv_sec = TIMEOUT_SOCKET;
        tv.tv_usec = 0;

        if(setsockopt(llisten, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv)) == -1){
            perror("Errore nell'installare il timeout per attendere il Bot");
            goto exit;
        }

        int bot_fd = accept(llisten, (struct sockaddr *)&bot, &lbot);

        if(bot_fd != -1 && bot.sin_addr.s_addr != htonl(INADDR_LOOPBACK)){
            printf("[*] Rilevato tentativo di connessione non previsto durante lo startup
del bot, chiudo la connessione\n");
            fflush(stdout);
            close(bot_fd);
        }
    }
}

```

bot_fd = -1; // impedisco così che mi entri nella if dopo e mi crei il thread
di gestione

```
}
tv.tv_sec = 0;

if(setsockopt(llisten, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv)) == -1){
    perror("Errore nel rimuovere il timeout per l'attesa del Bot");
    goto exit;
}

if(bot_fd != -1){

    client_arg *cbot = malloc(sizeof(client_arg));
    if(cbot == NULL){
        perror("Errore nella malloc");
        goto exit;
    }

    cbot->client_fd = bot_fd;
    cbot->client_addr = bot;
    cbot->sem_id = current_player;

    pthread_t tid;
    pthread_mutex_lock(&stato.lock);
    stato.active_threads++;
    pthread_mutex_unlock(&stato.lock);

    if(pthread_create(&tid, NULL, client_thread, cbot) != 0){
        perror("Errore nella creazione del thread di gestione del bot");
        pthread_mutex_lock(&stato.lock);
        stato.active_threads--;
        pthread_mutex_unlock(&stato.lock);
        close(bot_fd);
        free(cbot);
        goto exit;
    }
    pthread_detach(tid);
    current_player++;

} else perror("Errore nell'accept del bot");
}

slot = current_player; // connessioni accettate e non handshake completati
close(llisten);
llisten = -1;

if(slot > 0 && !shutdown_flag){

    for(int j = 0; j<slot; j++){
        semctl(sem1, j, SETVAL, 0);
    }

    printf("[*] Lobby chiusa, ricezione nuove richieste di partecipazione bloccate\n");
    fflush(stdout);

    struct sembuf sem;
    sem.sem_flg = 0;
    sem.sem_op = -slot; // -slot così mi fanno le post e torna a 0
```

```

sem.sem_num = 0;

try:
    if ((ret = semop(sem2, &sem, 1)) == -1 && errno != EINTR){
        perror("Errore nella semop");
        goto exit;
    } else if (ret == -1) {
        if (shutdown_flag) goto exit;
        goto try;
    }

    printf("[*] Startup della partita\n");
    fflush(stdout);

    struct sembuf startup;
    startup.sem_flg = 0;
    startup.sem_op = 1;
    int primo = -1; // lo uso per trovare il primo giocatore vivo a cui dare il gettone

    pthread_mutex_lock(&stato.lock);
    for(int j = 0; j < slot; j++){
        // ricerco il primo giocatore disponibile da cui far partire la partita
        player *p = trova_giocatore(j, true);
        if(p != NULL && p->alive) {
            primo = j;
            break;
        }
    }
    pthread_mutex_unlock(&stato.lock);

    if(primo == -1){
        printf("[!] Impossibile trovare un giocatore disponibile per lo startup\n");
        fflush(stdout);
        goto exit;
    } else startup.sem_num = primo;

start:
    if((ret = semop(sem1, &startup, 1)) == -1 && errno != EINTR){
        perror("Errore nell'avviare i player");
        goto exit;
    } else if (ret == -1) {
        if(shutdown_flag) goto exit;
        goto start;
    }

    printf("[*] Partita avviata con successo, player con sem_num = %d abilitato, vado in
background \n", primo);
    fflush(stdout);
} else {
    if(shutdown_flag) printf("\n[!] Richiesta di chiusura da segnalazione, partita non
avviata!\n");
    else if(!timeout) printf(" \n[!] Timer della lobby scaduto, nessun client
connesso!\n");
    else printf(" \n[!] Nessun client ha effettuato connessioni al server, chiudo...\n");
    fflush(stdout);
}

while(!shutdown_flag){
    pthread_mutex_lock(&stato.lock);
    int attivi = stato.active_threads;

```

```

        pthread_mutex_unlock(&stato.lock);

        if (attivi == 0) break;

        sleep(1);
    }

exit:
    if(shutdown_flag) printf("\n[*] Ricevuto segnale numero %d, routine di chiusura
avviata...\n", (int)sig_ricevuta);
    else printf("\n[*] Routine di chiusura avviata. Pulizia risorse in corso...\n");
    fflush(stdout);

    if (llisten >= 0) close(llisten);

    if (sem1 >= 0) semctl(sem1, 0, IPC_RMID);

    if (sem2 >= 0) semctl(sem2, 0, IPC_RMID);
    // devo attendere che tutti i thread terminino
    bool terminati = false;
    for(int max = 0; max< TIMEOUT_SHUTDOWN; max++){
        pthread_mutex_lock(&stato.lock);
        if(stato.active_threads == 0){
            terminati = true;
            pthread_mutex_unlock(&stato.lock);
            break;
        } else pthread_mutex_unlock(&stato.lock);
        sleep(1);
    }

    pthread_mutex_lock(&stato.lock);
    if (terminati){
        player *curr = stato.head, *temp;
        while (curr != NULL) {
            temp = curr;
            curr = curr->next;
            free(temp);
        }
        stato.head = NULL;
    } else printf("[!] Ci sono ancora %d thread attivi non terminati!\n",
stato.active_threads);

    pthread_mutex_unlock(&stato.lock);

    printf("[*] Server chiuso correttamente!\n");
    return 0;
}

void *client_thread(void *args){
    bool posted = false, afk = false; // mi segnala se ho fatto post su sem2
    client_arg *cargs = (client_arg *)args;
    player *me = NULL;
    int fd = cargs->client_fd;
    int ret;

    //recupero ip e porta del client collegato
    char ip[16];
    strncpy(ip, inet_ntoa(cargs->client_addr.sin_addr), 15);
    ip[15] = '\0';
    int portc = ntohs(cargs->client_addr.sin_port);

```



```

printf("[*] Nuova connessione da parte del client: %s:%d \n", ip, portc);
fflush(stdout);

azioni msg;
memset(&msg, 0, sizeof(msg));

if(recv_msg(cargs->client_fd, &msg) != 0 || msg.type != JOIN){
    printf("[!] Handshake non validato per %s:%d\n", ip, portc);
    fflush(stdout);
    goto exit;
    return NULL;
}

msg.username[USERNAME -1] = '\0';

me = add_player(fd, msg.username, carg->sem_id);

if (me == NULL) {
    printf("[!] Impossibile creare struct player per %s:%d\n", ip, portc);
    fflush(stdout);
    goto exit;
}

azioni welcome_msg;
memset(&welcome_msg, 0, sizeof(welcome_msg));
welcome_msg.type = WELCOME;
welcome_msg.player_id = me->id;

if(send_msg(cargs->client_fd, &welcome_msg) == -1){
    printf("[!] Errore nell'invio del messaggio di benvenuto a %s:%d\n", ip, portc);
    fflush(stdout);
    goto exit;
}

printf("[*] Giocatore %d (%s) connesso da %s:%d\n", me->id, msg.username, ip, portc);
fflush(stdout);

azioni gamemode;
memset(&gamemode, 0, sizeof(gamemode));
if(recv_msg(fd, &gamemode) != 0 || gamemode.type != MODE){
    printf("[!] Errore nella ricezione della gamemode dal client %s:%d\n", ip, portc);
    fflush(stdout);
    goto exit;
}

if(gamemode.gamemode != MODE_DEFAULT && gamemode.gamemode != MODE_1V1){
    printf("[!] Ricevuta una modalità di gioco non valida da %s:%d\n", ip, portc);
    fflush(stdout);
    goto exit;
}

pthread_mutex_lock(&stato.lock);

if(!stato.mode_scelta){
    stato.modalita = gamemode.gamemode;
    stato.mode_scelta = 1;
    printf("[*] Modalità di gioco impostata da %s:%d a %s\n", ip, portc, stato.modalita ==
MODE_1V1 ? "1v1" : "Default (TcT)");
    fflush(stdout);
}

```

```

}
bool rifiutato = (stato.modalita == MODE_1V1 && stato.count > 2);

pthread_mutex_unlock(&stato.lock);

if(rifiutato){
    printf("[!] Connessione rifiutata per %s:%d in quanto si è in 1v1 e ci sono già 2
client connessi\n", ip, portc);
    fflush(stdout);
    goto exit;
}

printf("[*] In attesa della formazione da %s:%d (ID: %d)\n", ip, portc, me->id);
fflush(stdout);

struct sembuf ready;
ready.sem_flg = 0;
ready.sem_num = 0;
ready.sem_op = 1;

if(ricezione_navi(fd, me) != 0){
    printf("[!] Formazione ricevuta non valida\n");
    fflush(stdout);
    pthread_mutex_lock(&stato.lock);
    me->alive = 0;
    pthread_mutex_unlock(&stato.lock);
    goto post;
}

printf("[*] Formazione ricevuta correttamente (%d, %s)!\n", me->id, me->username);
fflush(stdout);

bool fine_partita;

post:
    if((ret = semop(sem2, &ready, 1)) == -1 && errno != EINTR){
        if(errno != EIDRM && errno != EINVAL) perror("Impossibile segnalare l'avvenuta
ricezione in sem2");
        goto exit;
    } else if(ret == -1) {
        if(shutdown_flag) goto exit;
        goto post;
    }
    posted = true;
    pthread_mutex_lock(&stato.lock);
    bool vivo = me->alive;
    pthread_mutex_unlock(&stato.lock);

    if(!vivo) goto exit; // esco se sono stato marcato morto causa (almeno qui) formazione
invalida
    while(1){
        struct sembuf sem;
        sem.sem_flg = 0;
        sem.sem_num = me->sem_id;
        sem.sem_op = -1;
        bool colpito = false;

gback:
        if((ret = semop(sem1, &sem, 1)) == -1 && errno != EINTR){
            if(errno != EIDRM && errno != EINVAL) perror("Errore nel prendere il gettone");

```

```

        goto exit;
    } else if(ret == -1){
        if(shutdown_flag) goto exit;
        goto gback;
    }
    afk = false;

    pthread_mutex_lock(&stato.lock);
    fine_partita = stato.fine;
    pthread_mutex_unlock(&stato.lock);

    if(fine_partita) goto exit;

    pthread_mutex_lock(&stato.lock);
    int id = me->id;
    int alive = me->alive;
    int semid = me->sem_id;
    char username[USERNAME];
    strncpy(username, me->username, USERNAME);
    pthread_mutex_unlock(&stato.lock);

    if(!alive) goto next_turn;

    pthread_mutex_lock(&stato.lock);
    for(player *p = stato.head; p != NULL; p = p->next){
        if(p->id != id && p->alive){
            azioni infopk;
            memset(&infopk, 0, sizeof(infopk));
            infopk.type = INFO;
            infopk.player_id = p->id;
            strncpy(infopk.username, p->username, USERNAME-1);
            if(send_msg(me->socket, &infopk) == -1){
                printf("[!] Errore nel comunicare a %s (%d) la lista degli utenti
disponibili\n", username, id);
                fflush(stdout);
                me->alive = 0;
                pthread_mutex_unlock(&stato.lock);
                goto next_turn;
            }
        }
    }
}

pthread_mutex_unlock(&stato.lock);

azioni turno;
memset(&turno, 0, sizeof(turno));
turno.type = TURN;
turno.player_id = id;
if(send_msg(fd, &turno) == -1){
    printf("[!] Errore nel comunicare il turno a %s, rimuovo il player\n", username);
    fflush(stdout);
    pthread_mutex_lock(&stato.lock);
    me->alive = 0;
    pthread_mutex_unlock(&stato.lock);
    goto next_turn;
}
afk = true;
struct timeval tv;
tv.tv_sec = AFK_TIMEOUT;
tv.tv_usec = 0;

```

```

    if(setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv)) == -1){
        perror("Errore nell'installare il timeout di inattività sul socket del client");
        pthread_mutex_lock(&stato.lock);
        me->alive = 0;
        pthread_mutex_unlock(&stato.lock);
        goto next_turn;
    }

    azioni mosca;
    errno = 0; // levo errori residui
    if((ret = recv_msg(fd, &mosca)) == -1 || mosca.type != MOVE){
        if(ret == -1){
            afk = (errno == EAGAIN || errno == EWOULDBLOCK); // questi errori perchè se
            // scade il timeout senza ricevere dati, restituisce uno di questi errori
            if(afk) printf("[!] %s (%d) inattivo da %ds, rimuovo il player\n", username,
            id, AFK_TIMEOUT);
            else printf("[!] Connessione persa con %s (%d), rimuovo il player\n",
            username, id);
        } else {
            afk = false;
            printf("[!] Ricevuto un pacchetto dati non valido da %s (%d)\n", username,
            id);
        }
        fflush(stdout);
        pthread_mutex_lock(&stato.lock);
        me->alive = 0;
        pthread_mutex_unlock(&stato.lock);
        goto next_turn;
    }
    afk = false;
    tv.tv_sec = 0;
    if(setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv)) == -1){
        perror("Errore nella disattivazione del timer per il socket del client connesso");
        pthread_mutex_lock(&stato.lock);
        me->alive = 0;
        pthread_mutex_unlock(&stato.lock);
        goto next_turn;
    }

    azioni esito;
    memset(&esito, 0, sizeof(esito));
    esito.player_id = id;
    esito.affondata = NESSUNA_NAVE;
    esito.target_id = mosca.target_id;
    esito.x = mosca.x;
    esito.y = mosca.y;
    bool eliminato = false;

    pthread_mutex_lock(&stato.lock);
    player *bersaglio = trova_giocatore(mosca.target_id, false);

    // poichè uint8_t è unsigned, se passo un valore negativo ottengo sicuramente un
    // valore ≥ a GRID_SIZE quindi il controllo sul negativo
    // è superfluo
    if(bersaglio == NULL || !bersaglio->alive ){
        printf("[*] Bersaglio non più nella partita\n");
        fflush(stdout);
        esito.type = MISS;
    } else if ( mosca.x >= GRID_SIZE || mosca.y >= GRID_SIZE || mosca.target_id == id){

```

```

    printf("[!] Coordinate o bersaglio non validi per %s (%d)\n", username, id);
    fflush(stdout);
    esito.type = MISS;
} else {
    char cella = bersaglio->griglia[mossa.x][mossa.y];
    colpito = (cella >= '0' && cella <= '4');

    if(colpito){
        bersaglio->griglia[mossa.x][mossa.y] = 'X';
        esito.type = HIT;
        colpito = true;
        bool affondata = true;
        for(int i = 0; i<GRID_SIZE; i++){
            for(int j = 0; j<GRID_SIZE; j++){

                if(bersaglio->griglia[i][j] == cella){ // se trovo un altro elemento
di quella nave, vuol dire che ancora non è affondata
                    affondata = false;
                    break;
                }
            }
            if(!affondata) break; // nel caso abbia già trovato una nave esco
        }

        if(affondata){
            bersaglio->navi_rimaste--;
            esito.affondata = (uint8_t)(cella - '0'); // operazione inversa al
riempimento delle cella dove si faceva indice + '0'
            printf("[*] Il giocatore %s (%d) ha affondato la nave di %s (%d)!\n",
username, id, bersaglio->username, bersaglio->id);
            fflush(stdout);

            if(bersaglio->navi_rimaste <= 0){
                bersaglio->alive = 0;
                eliminato = true;
            }

        }

    } else {
        esito.type = MISS;
        if(bersaglio->griglia[mossa.x][mossa.y] == '~') {
            bersaglio->griglia[mossa.x][mossa.y] = '0';
        }
    }
}

broadcast(&esito);

int bersaglio_id = -1;
if(bersaglio != NULL) bersaglio_id = bersaglio->id;

pthread_mutex_unlock(&stato.lock);

if(eliminato){
    azioni gameover;
    memset(&gameover, 0, sizeof(gameover));
    gameover.type = ELIMINATED;
    gameover.target_id = bersaglio_id;
}

```

```

        pthread_mutex_lock(&stato.lock);
        broadcast(&gameover);
        pthread_mutex_unlock(&stato.lock);
    }

    // poichè a priori non so se il prossimo giocatore è stato eliminato, devo trovare il
    prossimo ancora vivo e poi passare l'id alla sembuf
next_turn:
    pthread_mutex_lock(&stato.lock);
    int prossimo = -1;
    int test = semid;
    for(int i = 0; i < slot; i++){
        test = (test + 1) % slot;
        if (test == semid) continue;
        player *p = trova_giocatore(test, true); // true specifica se ricerca per numero
di semaforo, false solo con id
        if(p != NULL && p->alive){
            prossimo = test;
            break;
        }
    }
    int vivi = 0;
    int vincitore = id;
    for(player *p = stato.head; p != NULL; p = p->next){
        if(p->alive){
            vivi ++;
            vincitore = p->id;
        }
    }

    bool solo = (vivi <= 1 || prossimo == -1); // per vedere se ci sono altri avversari
    if(solo && !stato.fine){
        stato.fine = fine_partita = 1; //sono solo chiudo la partita
    } else if(colpito && !solo) prossimo = semid;

    pthread_mutex_unlock(&stato.lock);

    if(fine_partita){
        azioni win;
        memset(&win, 0, sizeof(win));
        win.type = WIN;
        win.player_id = vincitore;

        pthread_mutex_lock(&stato.lock);
        broadcast(&win);
        pthread_mutex_unlock(&stato.lock);

        // devo risvegliare tutti i thread in lock
        for(int j = 0; j < slot; j++){
            struct sembuf sem;
            sem.sem_op = 1;
            sem.sem_flg = 0;
            sem.sem_num = j;

wake:
            if((ret = semop(sem1, &sem, 1)) == -1 && errno != EINTR){
                if(errno != EIDRM && errno != EINVAL) perror("Errore nel risvegliare tutti
i thread per fine partita");
                goto exit;
            } else if (ret == -1) {

```

```

        if(shutdown_flag) goto exit;
        goto wake;
    }
}

goto exit;

} else if (prossimo == -1) goto exit;
sem.sem_op = 1;
sem.sem_num = prossimo;

pass:
    if((ret = semop(sem1, &sem, 1)) == -1 && errno != EINTR){
        if (errno != EIDRM && errno != EINVAL) perror("Errore nel rilasciare il gettone
del semaforo al prossimo player");
        goto exit;
    } else if (ret == -1) {
        if(shutdown_flag) goto exit;
        goto pass;
    }

    pthread_mutex_lock(&stato.lock);
    alive = me->alive;
    pthread_mutex_unlock(&stato.lock);
    if(!alive) goto exit;
}

exit:
    if(!posted){
        struct sembuf unlock;
        unlock.sem_num = 0;
        unlock.sem_flg = 0;
        unlock.sem_op = 1;
    exit_post:
        if((ret = semop(sem2, &unlock, 1)) == -1 && errno != EINTR){
            if (errno != EIDRM && errno != EINVAL) perror("Errore nel segnalare la presenza su
sem2");
        } else if (ret == -1) if(!shutdown_flag) goto exit_post;
        posted = true;
    }
    pthread_mutex_lock(&stato.lock);
    stato.active_threads--;

    if(me != NULL){
        remove_player(me->id, afk);
    }
    pthread_mutex_unlock(&stato.lock);

    free(cargs);
    close(fd);
    pthread_exit(NULL);
}

void *add_player(int fd, char *username, int sem_id){

    // serve per aggiungere un nuovo player, non necessita di lock su mutex stato.lock perchè
viene già fatto qui dentro
    player *new_player = malloc(sizeof(player));

```

```

    if(new_player == NULL){
        perror("Errore nella malloc del nuovo giocatore");
        return NULL;
    }

    for(int i = 0; i < GRID_SIZE; i++){
        for(int j = 0; j < GRID_SIZE; j++){
            new_player->griglia[i][j] = '~';
        }
    }

    strncpy(new_player->username, username, USERNAME - 1);
    new_player->username[USERNAME-1] = '\0';
    new_player->socket = fd;
    new_player->alive = 1;

    new_player->navi_rimaste = SHIP_NUMBER;

    pthread_mutex_lock(&stato.lock);

    stato.next_id++;
    new_player->id = stato.next_id;
    new_player->sem_id = sem_id;

    new_player->next = stato.head;
    stato.head = new_player;
    stato.count++;

    pthread_mutex_unlock(&stato.lock);

    return new_player;
}

void remove_player(int id, bool flag){
    // serve per rimuovere i giocatori in fase di chiusura o per altri scenari
    player *curr = stato.head;
    player *prev = NULL;

    while(curr != NULL){
        if(curr->id == id){
            if(prev == NULL){
                stato.head = curr->next;
            } else {
                prev->next = curr->next;
            }
            stato.count--;
            printf("[*] Giocatore %d (%s) rimosso con successo%s", curr->id, curr->username,
flag ? " per inattività!\n" : "!\n");
            fflush(stdout);
            free(curr);
            return;
        }
        prev = curr;
        curr = curr->next;
    }
    return;
}

player *trova_giocatore(int valore, bool flag){

```



```

    // permette di trovare un giocatore tramite id (con flag = false) oppure sem_id (con flag
= true)
    player *current = stato.head;
    int chiave; // scelgo la chiave in base a quel che gli passo

    while(current != NULL){
        chiave = flag ? current->sem_id : current->id;

        if(chiave == valore) return current;
        current = current->next;
    }

    return NULL;
}

void *udp_discovery_port(void *args){
    unsigned int porta = *((unsigned int *)args);
    int socket_fd = socket(AF_INET, SOCK_DGRAM, 0), letti;

    if(socket_fd < 0){
        perror("Errore nella creazione del socket UDP");
        pthread_exit(NULL);
    }

    struct timeval time; // dal man di setsockopt
    time.tv_sec = TIMEOUT_SOCKET;
    time.tv_usec = 0;
    if (setsockopt(socket_fd, SOL_SOCKET, SO_RCVTIMEO, &time, sizeof(time)) < 0) {
        perror("Errore nel setup del timer in setsockopt");
        close(socket_fd);
        pthread_exit(NULL);
    }

    struct sockaddr_in udp_addr, client_addr;
    socklen_t client_addr_len = sizeof(client_addr); // serve per dopo nelle send e recv

    //ripulisco le strutture per sicurezza
    memset(&udp_addr, 0, sizeof(udp_addr));
    memset(&client_addr, 0, sizeof(client_addr));

    udp_addr.sin_family = AF_INET;
    udp_addr.sin_port = htons(DISCOVERY_PORT);
    udp_addr.sin_addr.s_addr = htonl(INADDR_ANY);

    if(bind(socket_fd, (struct sockaddr *)&udp_addr, sizeof(udp_addr))<0){
        perror("Errore nel bind del socket UDP");
        close(socket_fd);
        pthread_exit(NULL);
    }

    char buffer[BUFFER_SIZE], reply[BUFFER_SIZE];

    while(lobby_aperta){
        buffer[0] = '\0';
        letti = recvfrom(socket_fd, buffer, sizeof(buffer)-1, 0, (struct sockaddr
*)&client_addr, &client_addr_len); // bloccata al massimo per 2s (TIMEOUT_SOCKET)
        if(letti > 0){
            buffer[letti] = '\0';
            if(strcmp(buffer, "DISCOVER") == 0){
                snprintf(reply, sizeof(reply), "%u", porta);
            }
        }
    }
}

```

```

        sendto(socket_fd, reply, strlen(reply), 0, (struct sockaddr *)&client_addr,
client_addr_len);
        printf("[*] Ricevuta richiesta di discovery da %s:%d\n",
inet_ntoa(client_addr.sin_addr), ntohs(client_addr.sin_port));
        fflush(stdout);
    }
    } else if(letti < 0){
        if(errno == EINTR || errno == EAGAIN || errno == EWOULDBLOCK) continue; // se
l'errore è dovuto a una segnalazione, riprovo
        perror("Errore nella ricezione del messaggio UDP");
    }
}

close(socket_fd);
pthread_exit(NULL);
}

bool validazione_formazione(char board[GRID_SIZE][GRID_SIZE], int x, int y, char orientazione,
uint8_t dimensione_nave, int indice){
    // la funzione si occupa di controllare e validare il piazzamento andando a controllare i
limiti di griglia
    // marcando quale posizione risulta occupata, non è il controllo ufficiale ma serve
solo in fase di posizionamento
    // il vero controllo poi lo rifarà anche il server.
    // funzione del tutto analoga a quella che c'è in client.c
    // indice indica quale nave è arrivata
    int delta_riga, delta_colonna, r, c;

    if ( orientazione == 'N' ) { delta_riga = -1; delta_colonna = 0;}
    else if ( orientazione == 'S' ) { delta_riga = 1; delta_colonna = 0;}
    else if ( orientazione == 'E' ) { delta_riga = 0; delta_colonna = 1;}
    else if ( orientazione == 'O' ) { delta_riga = 0; delta_colonna = -1;}
    else return false;

    for (int i = 0; i < dimensione_nave; i++) {
        r = x + i * delta_riga;
        c = y + i * delta_colonna;

        if (r < 0 || r >= GRID_SIZE || c < 0 || c >= GRID_SIZE) {
            return false;
        } else if (board[r][c] != '~') {
            return false;
        }
    }

    for ( int i = 0; i < dimensione_nave; i++ ) {
        r = x + i * delta_riga;
        c = y + i * delta_colonna;
        board[r][c] = '0' + indice;
    }
    return true;
}

int ricezione_navi (int fd, player *me){
    // azzerare la griglia dell'utente e ricevere la formazione
    for(int i = 0; i<GRID_SIZE; i++){
        for (int j = 0; j<GRID_SIZE; j++){
            me->griglia[i][j] = '~';
        }
    }
}

```

```

    int index /*della nave*/, x, y;
    char orientazione;
    for(int i = 0; i<SHIP_NUMBER; i++){
        posizionamento p;
        memset(&p, 0, sizeof(p));
        if(readn(fd, &p, sizeof(p)) != (ssize_t)sizeof(p)){ //readn mi da ssize_t, sizeof mi
da size_t: per coerenza sistema i cast
            perror("Errore nella ricezione della formazione");
            return -1;
        }

        index = p.index;
        x = p.x; // non servono ntohl poichè lavorano su int, non su byte singolo, endianness
safe in quanto solo un byte (endianness a livello di byte)
        y = p.y;
        orientazione = p.orientation;

        if(index != i){
            printf("[!] Indice della nave errato\n");
            fflush(stdout);
            return -1;
        }

        if(!validazione_formazione(me->griglia, x, y, orientazione, ship_type[i].size, i)){
            printf("[!] Inserimento della nave non valido\n");
            fflush(stdout);
            return -1;
        }

    }

    return 0;
}

void broadcast(azioni *esito){
    // serve per le comunicazioni di esiti a tutti i player, da utilizzare sotto mutex di
stato.lock
    for(player *p = stato.head; p != NULL; p = p->next){
        if(send_msg(p->socket, esito) == -1){
            printf("[!] Impossibile inviare il messaggio di esito al giocatore %s (%d)\n",
p->username, p->id);
            fflush(stdout);
        }
    }
}

/*
    INSIEME DELLE FUNZIONI DEDITE ALLA RICEZIONE DI SEGNALAZIONI
*/

void timeout_lobby(int sig){
    timeout = 0;
}

void chiusura (int sig){
    sig_ricevuta = sig;
    shutdown_flag = 1;
}

```

```
void gestore_bot(int sig){
    wait(NULL);
}
```

client.c

```
/*
    CLIENT POSIX/WINAPI RELEASE
*/

#include "../..//protocollo/protocollo.h"
#include "../..//gui/gui.h"
#include "../..//utils/utils.h"

#include <unistd.h>
#include <errno.h>
#include <signal.h>
#include <sys/types.h>
#include <ctype.h> // semplifica la validazione dell'orientazione quando ricevo la flotta
#include <fcntl.h>

#ifdef _WIN32
    #include <sys/mman.h>
    #include <sys/socket.h>
    #include <netinet/in.h>
    #include <netdb.h>
    #include <arpa/inet.h>
#else
    #include <windows.h>
#endif

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>

volatile sig_atomic_t shutdown_flag = 0;

#ifdef _WIN32
    BOOL WINAPI ctrl_handler(DWORD ctrl_type);
    static void chiudi_socket(SOCKET fd); // centralizzo la chiusura del socket e mi
    semplifica anche il passaggio a WINAPI
    // standard
    WINAPI
#else
    void gestore(int sig);
    static void chiudi_socket(int fd); // centralizzo la chiusura del socket e mi
    semplifica anche il passaggio a WINAPI
#endif

int discovery_server(char *ip, unsigned int *port); // serve qualora non passo argomenti in
esecuzione o qualora il server non sia raggiungibile con i parametri specificati
int connetti(char *ip, unsigned int porta, struct sockaddr_in *server_addr);
int piazzamento_navi (int socket);
int invio_navi(int socket, posizionamento *navi);
```

```

int getUsername(char *buf, size_t len); // per prendere l'username del nuovo giocatore
bool validazione( bool board[GRID_SIZE][GRID_SIZE], int x, int y, char orientazione, uint8_t
dimensione_nave ); // valido la formazione (se è nei limiti prima di inviare)

unsigned int port;

#ifdef _WIN32
    SOCKET socketfd = INVALID_SOCKET;
#else
    int socketfd = -1;
#endif

int main(int argc, char **argv) {

#ifdef _WIN32
    WSADATA wsa;

    bool aperto = false;
    if(WSAStartup(MAKEWORD(2,2), &wsa) != 0){
        fprintf(stderr,"Errore nello startup del socket (WINAPI)\n");
        return -1;
    }

    aperto = true;
    HANDLE hout = GetStdHandle(STD_OUTPUT_HANDLE);
    DWORD console_mode = 0;
    if(hout != INVALID_HANDLE_VALUE && GetConsoleMode(hout, &console_mode))
SetConsoleMode(hout, console_mode|ENABLE_VIRTUAL_TERMINAL_PROCESSING);

    if(!SetConsoleCtrlHandler(ctrl_handler, TRUE)){
        fprintf(stderr,"Errore nell'installazione del gestore per il Ctrl+C
(WINAPI)\n");
        WSACleanup();
        return -1;
    }
#else
    struct sigaction sa;
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = gestore;
    sa.sa_flags = 0;

    if(sigaction(SIGINT, &sa, NULL) == -1 || sigaction(SIGTERM, &sa, NULL) == -1){
        fprintf(stderr, "Errore nell'installazione della sigaction\n");
        goto chiusura;
    }

    sa.sa_handler = SIG_IGN;
    if(sigaction(SIGPIPE, &sa, NULL) == -1){
        fprintf(stderr, "Errore nell'installazione della sigaction per la sigpipe\n");
        goto chiusura;
    }
#endif

    char ip_buf[16] = {0};
    port = 0;
    bool auto_mode = false;

    if(argc == 1){

```

```

        auto_mode = true;
        gui_init(auto_mode);
        if(discovery_server(ip_buf, &port) == -1){
            discovery_server_errore(argv[0]);
            goto chiusura;
        }
    } else if(argc < 3){
        sintassi_corretta(argv[0]);
        goto chiusura;
    } else {
        int tport = atoi(argv[2]);
        if(tport < 5000 || tport > 65535){
            auto_mode = true;
            gui_init(auto_mode);
            invalid_port();
            if(discovery_server(ip_buf, &port) == -1){
                discovery_server_errore(argv[0]);
                goto chiusura;
            }
        } else {
            port = (unsigned int)tport;
            strncpy(ip_buf, argv[1], sizeof(ip_buf) - 1);
            gui_init(auto_mode);
        }
    }
}

struct sockaddr_in server_addr;
memset(&server_addr, 0, sizeof(server_addr));

int mio_id = -1;

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(port);

socketfd = connetti(ip_buf, port, &server_addr);
if(socketfd == -1){
    connection_lost_fallback(ip_buf, port);
    if(discovery_server(ip_buf, &port) == -1){
        server_not_found();
        goto chiusura;
    }

    socketfd = connetti(ip_buf, port, &server_addr);
}

if(socketfd == -1){
    connection_error();
    goto chiusura;
}

char username[USERNAME];
if(getUsername(username, USERNAME) == -1) goto chiusura;

//procedimento di handshake con il server
azioni msg;
memset(&msg, 0, sizeof(msg));
strcpy(msg.username, username, USERNAME-1);
msg.username[USERNAME - 1] = '\0';
msg.type = JOIN;

```

```

if(send_msg(socketfd, &msg) == -1){
    fprintf(stderr, "errore nell'invio del messaggio di JOIN\n");
    goto chiusura;
}

azioni welcome_msg;
if(recv_msg(socketfd, &welcome_msg) == -1){
    fprintf(stderr, "errore nella ricezione del messaggio di WELCOME\n");
    goto chiusura;
}

if(welcome_msg.type != WELCOME){
    fprintf(stderr, "messaggio di benvenuto non valido\n");
    goto chiusura;
}

// ok handshake, il server mi ha assegnato un id
mio_id = welcome_msg.player_id;
server_connected(ip_buf, port, mio_id);

azioni mode_msg;
memset(&mode_msg, 0, sizeof(mode_msg));
mode_msg.type = MODE;
mode_msg.player_id = mio_id;
int gmode = game_mode();
if(shutdown_flag) goto chiusura;
mode_msg.gamemode = gmode;

if(send_msg(socketfd, &mode_msg) == -1){
    fprintf(stderr, "errore nell'invio della modalità di gioco\n");
    goto chiusura;
}

init_board();
if(piazzamento_navi(socketfd) == -1) goto chiusura;

waiting_player();

azioni pck;
bool vivo = true;
bool fine = false;
char esito;

while(!fine && !shutdown_flag){
    memset(&pck, 0, sizeof(pck));
    if(recv_msg(socketfd, &pck) == -1){
        if(!shutdown_flag) connection_lost(); // mi serve in quanto può dare -1
        anche per ctrl+c
        break;
    }

    switch(pck.type){
        case INFO:
            bersagli(pck.player_id, pck.username);
            break;

        case TURN:
            if(pck.player_id == mio_id && vivo){
                turno();
            }
    }
}

```

```

        azioni_mossa;
        memset(&mossa, 0, sizeof(mossa));
        mossa.player_id = mio_id;

        ricezione_mossa(&mossa);
        if(shutdown_flag) goto chiusura;
        if(send_msg(socketfd, &mossa) == -1){
            errore_invio_mossa();
            connection_lost();
            goto chiusura;
        }

    } else {
        non_mio_turno(pck.player_id);
    }
    break;

case HIT:
case MISS:
    if(pck.player_id == mio_id){
        if(pck.type == HIT){
            esito = 'X';
        } else esito = 'O';

        target_grids[pck.target_id][pck.x][pck.y] = esito;
        targetId = pck.target_id;

        if(pck.type == HIT){
            if (pck.affondata < SHIP_NUMBER)
                nave_affondata(ship_type[pck.affondata].name, pck.target_id);
            else colpito();
        } else miss();

        clean_screen();
        draw_grids();
    } else if(pck.target_id == mio_id){

        if(pck.type == MISS){
            esito = 'O';
        } else esito = 'X';

        grid[pck.x][pck.y] = esito;

        if(pck.type == HIT){
            if (pck.affondata < SHIP_NUMBER)
                s_nave_affondata(ship_type[pck.affondata].name, pck.player_id);
            else colpito();
        } else miss();

        clean_screen();
        draw_grids();

    } else {
        spettatore(&pck);
    }
    break;

case ELIMINATED:

```



```

        if(pck.target_id == mio_id){
            vivo = false;
            s_eliminato();
        } else {
            eliminato(pck.target_id);
        }
        break;

    case WIN:
        if(pck.player_id == mio_id){
            s_vittoria();
        } else {
            vittoria(pck.player_id);
        }
        fine = true;
        break;
    default:
        break;
}

}

chiusura:

    if(shutdown_flag){
        chiusura_forzata();
        fflush(stdout);
    }

    if(socketfd != -1){
        chiudi_socket(socketfd);
        socketfd = -1;
    }
#ifdef _WIN32
    if(aperto) WSACleanup();
#endif

    close_game();

    return 0;
}

int connetti(char *ip, unsigned int porta, struct sockaddr_in *server_addr){
    memset(server_addr, 0, sizeof(struct sockaddr_in));
    server_addr->sin_family = AF_INET;
    server_addr->sin_port = htons(porta);

#ifdef _WIN32
    if (inet_pton(AF_INET, ip, &server_addr->sin_addr) != 1) {
        invalid_ip(ip);
        return -1;
    }
#else
    if (inet_aton(ip, &server_addr->sin_addr) == 0) {
        invalid_ip(ip);
        return -1;
    }
#endif
}

```

```

#ifdef _WIN32
    SOCKET sfd = socket(AF_INET, SOCK_STREAM, 0);
#else
    int sfd = socket(AF_INET, SOCK_STREAM, 0);
#endif

    if(sfd == -1){
        fprintf(stderr, "Errore nell'apertura del socket di connessione\n");
        return -1;
    }

    if(connect(sfd, (struct sockaddr *)server_addr, sizeof(struct sockaddr_in)) == -1){
        fprintf(stderr, "Errore nella connessione al server\n");
        chiudi_socket(sfd);
        return -1;
    }

    return sfd;
}

int getUsername(char *buf, size_t len){

    // poichè causa portabilità di open non funziona su WINAPI, uso fopen
    FILE *fd = fopen("_user", "r");
    if(fd == NULL){
        if(errno != ENOENT) fprintf(stderr, "Errore nell'apertura del file di gestione
degli utenti\n");
        //se errno == ENOENT vuol dire che è la prima creazione del file
    } else {
        buf[0] = '\0';
        if(fgets(buf, len, fd) != NULL){
            size_t letti = strlen(buf);

            if(letti > 0 && buf[letti - 1] == '\n'){
                buf[letti-1] = '\0';
            }

            if(strlen(buf) > 0) {
                welcomeback(buf);
                fclose(fd);
                return 0;
            }
        }

        fclose(fd);
    }

    char temp[BUFFER_SIZE];
    while(1) {
        inserisci_username(len - 1);

        if(scanf("%255s", temp) == EOF){
            return -1;
        }
        fflush_stdin();

        if (strlen(temp) < len) {
            FILE *fd = fopen("_user", "w");
            if(fd == NULL){
                fprintf(stderr, "Errore nell'apertura del file\n");
            }
        }
    }
}

```

```

        mod_ospite();
        fprintf(stderr, "Procedo normalmente escludendo la scrittura sul
file \n");

        strcpy(buf, temp);
    } else {
        strcpy(buf, temp);
        fputs(buf, fd);
        fclose(fd);
    }
    break;
}
}
username_too_long(len -1);
}
return 0;
}

int piazzamento_navi (int socket) {

    posizionamento posizioni_navi[SHIP_NUMBER];
    int x, y = 0;
    char orientazione[2], riga_in[2];
    bool occupata[GRID_SIZE][GRID_SIZE] = {false}, valid; // griglia temporanea per capire
dove ho messo le navi
    int letti;

    for (int i = 0; i < SHIP_NUMBER; i++ ) {
        bool errore_pos = false;
        do {
            riga_in[0] = '?';
            y = 0;
            orientazione[0] = '?';
            clean_screen();
            mostra_flotta();
            if(errore_pos){
                invalid_placement();
                errore_pos = false;
            }
            inserisci_coordinate(ship_type[i].name, ship_type[i].size);
            while((letti = scanf("%1s %d %1s", riga_in, &y, orientazione))!= 3 ||
toupper(riga_in[0]) < 'A' || toupper(riga_in[0]) > 'A' + GRID_SIZE - 1 // -1 perchè sempre da
1-GRID_SIZE -1 come per le coordinate numeriche
                || (toupper(orientazione[0]) != 'N' &&
toupper(orientazione[0]) != 'S' && toupper(orientazione[0]) != 'E' && toupper(orientazione[0])
!= 'O' && toupper(orientazione[0]) != 'W')){
                if(letti == EOF) return -1;
                if(shutdown_flag) return -1;

                fflush_stdin();
                clean_screen();
                mostra_flotta();
                invalid_input(riga_in[0], y, orientazione[0]);
                inserisci_coordinate(ship_type[i].name, ship_type[i].size);
            }
            x = toupper(riga_in[0]) - 'A' + 1 ;
            orientazione[0] = (char)toupper(orientazione[0]);
            if (orientazione[0] == 'W') orientazione[0] = 'O';
            fflush_stdin();
            valid = validazione(occupata, x - 1, y - 1, orientazione[0],
ship_type[i].size);

```

```

        if (!valid) errore_pos = true;
    } while (!valid);

    posizioni_navi[i].index = i;
    posizioni_navi[i].x = x-1;
    posizioni_navi[i].y = y-1;
    posizioni_navi[i].orientation = orientazione[0];

    addboat(x - 1 , y - 1, orientazione[0], ship_type[i].size);

    clean_screen();
    posizionamento_ok(ship_type[i].name, toupper(riga_in[0]), y,
    toupper(orientazione[0]));
    }

    mostra_flotta();

    if(invio_navi(socket, posizioni_navi) == -1){
        if(errno == EPIPE || errno == ECONNRESET) connection_lost();
        else fprintf(stderr, "errore nell'invio delle posizioni delle navi\n");
        return -1;
    }

    return 0;
}

int invio_navi(int socket, posizionamento *navi){
    for(int i = 0; i< SHIP_NUMBER; i++){
        posizionamento p;
        memset(&p, 0, sizeof(p)); //pulisco prima di inviare
        p.index = navi[i].index;
        p.x = navi[i].x; // endianess safe -> mando 1 byte solo e non 4
        p.y = navi[i].y;
        p.orientation = navi[i].orientation;
        if(written(socket, &p, sizeof(posizionamento)) != sizeof(posizionamento)) return
-1;
    }
    return 0;
}

int discovery_server(char *ip, unsigned int *port){

#ifdef _WIN32
    SOCKET sock = socket(AF_INET, SOCK_DGRAM, 0);
#else
    int sock = socket(AF_INET, SOCK_DGRAM, 0);
#endif

    if(sock == -1){
        fprintf(stderr,"Errore nell'apertura del socket per l'UDP\n");
        return -1;
    }

    int abilitata = 1;
    if(setsockopt(sock, SOL_SOCKET, SO_BROADCAST, (const char *)&abilitata,
sizeof(abilitata))<0){
        fprintf(stderr,"Errore nell'abilitare la modalità broadcast sul socket di
discovery\n");
        chiudi_socket(sock);
        return -1;
    }
}

```

```

    }
#ifdef _WIN32
    DWORD timeout_sock = TIMEOUT_SOCKET * 1000;
    if (setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, (const char *)&timeout_sock,
sizeof(timeout_sock)) < 0) {
        fprintf(stderr,"errore nel setup del timer in setsockopt\n");
        chiudi_socket(sock);
        return -1;
    }
#else
    struct timeval time; // dal man di setsockopt
    time.tv_sec = TIMEOUT_SOCKET;
    time.tv_usec = 0;
    if (setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, &time, sizeof(time)) < 0) {
        fprintf(stderr,"errore nel setup del timer in setsockopt\n");
        chiudi_socket(sock);
        return -1;
    }
#endif

    struct sockaddr_in broadcast;
    memset(&broadcast, 0, sizeof(broadcast));
    broadcast.sin_family = AF_INET;
    broadcast.sin_port = htons(DISCOVERY_PORT);
    broadcast.sin_addr.s_addr = htonl(INADDR_BROADCAST);

    discovery_req_sent();

    struct sockaddr_in ricezione;
    socklen_t ricezione_s = sizeof(ricezione); // serve necessariamente perchè la recvfrom
vuole un puntatore alla dim della struttura (man)
    char buffer[BUFFER_SIZE];

    ssize_t n;

    for(int i = 0; i<TENTATIVI; i++){

        if(sendto(sock, DISCOVER, strlen(DISCOVER), 0, (struct sockaddr *)&broadcast,
sizeof(broadcast))<0){
            fprintf(stderr, "Errore nell'inviare il payload di discovery\n");
            chiudi_socket(sock);
            return -1;
        }

        n = recvfrom(sock, buffer, sizeof(buffer) -1 , 0, (struct sockaddr
*)&ricezione, &ricezione_s);
        if(shutdown_flag) break;
        if (n > 0) {
            buffer[n] = '\0';

            if (atoi(buffer) > 0) {
                strncpy(ip, inet_ntoa(ricezione.sin_addr), 15); // il server manda solo la
porta (Payload), il resto è nell'header del pacchetto
                ip[15] = '\0';
                *port = (unsigned int)atoi(buffer);

                server_found(ip, *port);
                chiudi_socket(sock);
                return 0;
            }
        }
    }

```

```

    }

    failed_attempt(i);

}

if(shutdown_flag) sig_received();
else server_not_found_attempt();
chiudi_socket(sock);
return -1;
}

// la validazione viene poi rifatta dal server
bool validazione( bool board[GRID_SIZE][GRID_SIZE], int x, int y, char orientazione, uint8_t
dimensione_nave ) {
    // la funzione si occupa di controllare e validare il piazzamento andando a
    controllare i limiti di griglia
    // marcando quale posizione risulta occupata, non è il controllo ufficiale ma serve
    solo in fase di posizionamento
    // il vero controllo poi lo rifarà anche il server.
    int delta_riga, delta_colonna, r, c;

    if ( orientazione == 'N' ) { delta_riga = -1; delta_colonna = 0;}
    else if ( orientazione == 'S' ) { delta_riga = 1; delta_colonna = 0;}
    else if ( orientazione == 'E' ) { delta_riga = 0; delta_colonna = 1;}
    else if ( orientazione == 'O' ) { delta_riga = 0; delta_colonna = -1;}
    else return false;

    for ( int i = 0; i < dimensione_nave; i++ ) {
        r = x + i * delta_riga;
        c = y + i * delta_colonna;
        if ( r < 0 || r >= GRID_SIZE || c < 0 || c >= GRID_SIZE ) {
            return false;
        } else if ( board[r][c] == true ) return false;
    }

    for ( int i = 0; i < dimensione_nave; i++ ) {
        r = x + i * delta_riga;
        c = y + i * delta_colonna;
        board[r][c] = true;
    }
    return true;
}

#ifdef _WIN32
static void chiudi_socket(SOCKET fd){
    closesocket(fd);
}

BOOL WINAPI ctrl_handler(DWORD sig){
    if(sig == CTRL_C_EVENT || sig == CTRL_BREAK_EVENT || sig == CTRL_CLOSE_EVENT){
        shutdown_flag = 1;
        if(socketfd != -1) chiudi_socket(socketfd);
        return TRUE;
    }
    return FALSE;
}

#else

```

```

static void chiudi_socket(int fd){
    close(fd);
}

void gestore(int sig){
    //(void)sig; scarta il valore della segnalazione, è superfluo quindi si può levare
    come mettere, non cambia nulla
    shutdown_flag = 1;
}
#endif

```

bot.c

```

/*
    BOT POSIX RELEASE
*/

#include "../protocollo/protocollo.h"
#include "../utils/utils.h"

#include <unistd.h>
#include <errno.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#include <arpa/inet.h>

#include <time.h>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <stdbool.h>
#include <stdint.h>

// non necessitando di interfaccia grafica, sarebbe stato inutile importare tutta la gui per
solo due colori
// li ridefinisco qui
#define CIANO "\x1b[36m"
#define RESET "\x1b[0m"

volatile sig_atomic_t shutdown_flag = 0;

void gestore(int sig);
int connetti(char *ip, unsigned int porta, struct sockaddr_in *server_addr);
int piazzamento_navi (int socket);
int invio_navi(int socket, posizionamento *navi);
bool validazione(bool board[GRID_SIZE][GRID_SIZE], int x, int y, char orientazione, int
dimensione_nave ); // valido la formazione (se è nei limiti prima di inviare)
void selezione_prossima_mossa(uint8_t *target, uint8_t *x, uint8_t *y); // permette di far
scegliere al bot la prossima mossa
void ricerca(uint8_t *x_out, uint8_t *y_out);
bool cella_papabile(int x, int y);

bool tentativi[GRID_SIZE][GRID_SIZE] = {false};

```

```

bool inseguimento = false;
int colpo_iniziale_x, colpo_iniziale_y; /* primo HIT ad aprire un inseguimento di una nave da
affondare */
int direzione_colpi = -1;
int direzione_tentata = -1;
int ultimo_colpo_x, ultimo_colpo_y; /* ultima cella colpita con successo durante un
inseguimento */
bool riprovato_verso_opposto = false; /* per sapere se, dopo un MISS in un verso, è già stato
provato il verso opposto */
int avversario_id = -1;
unsigned int port;

int main(int argc, char **argv) {

    if(argc < 3){
        fprintf(stderr, "[BOT] Sintassi corretta: %s <ip> <port>\n", argv[0]);
        return -1;
    }

    char ip_buf[16] = {0};
    int sockfd = -1;
    srand(time(NULL));

    strncpy(ip_buf, argv[1], sizeof(ip_buf) - 1);
    port = (unsigned int)atoi(argv[2]);

    struct sigaction sa;
    sigemptyset(&sa.sa_mask);
    sa.sa_handler = gestore;
    sa.sa_flags = 0;

    if(sigaction(SIGINT, &sa, NULL) == -1 || sigaction(SIGTERM, &sa, NULL) == -1){
        perror("Errore nell'installazione della sigaction");
        goto chiusura;
    }

    sa.sa_handler = SIG_IGN;
    if(sigaction(SIGPIPE, &sa, NULL) == -1){
        perror("Errore nell'installazione della sigaction per la SIGPIPE");
        goto chiusura;
    }

    struct sockaddr_in server_addr;
    memset(&server_addr, 0, sizeof(server_addr));

    int mio_id = -1;

    if((sockfd = connetti(ip_buf, port, &server_addr)) == -1){
        printf(CIANO"[BOT] Impossibile collegarsi al server\n"RESET);
        goto chiusura;
    }

    //procedimento di handshake con il server
    azioni msg;
    memset(&msg, 0, sizeof(msg));
    strncpy(msg.username, "BOT", USERNAME-1);
    msg.type = JOIN;

    if(send_msg(sockfd, &msg) == -1){
        printf(CIANO"[BOT] Errore nell'invio del messaggio di JOIN\n"RESET);
    }
}

```



```

        goto chiusura;
    }

    azioni welcome_msg;
    if(recv_msg(socketfd, &welcome_msg) == -1){
        printf(CIANO"[BOT] Errore nella ricezione del messaggio di WELCOME\n"RESET);
        goto chiusura;
    }

    if(welcome_msg.type != WELCOME){
        printf(CIANO"[BOT] Messaggio di benvenuto non valido\n"RESET);
        goto chiusura;
    }

    // ok handshake, il server mi ha assegnato un id
    mio_id = welcome_msg.player_id;
    printf(CIANO"[BOT] Connesso a %s:%u, id %d\n"RESET, ip_buf, port, mio_id);
    fflush(stdout);

    azioni mode_msg;
    memset(&mode_msg, 0, sizeof(mode_msg));
    mode_msg.type = MODE;
    mode_msg.player_id = mio_id;
    mode_msg.gamemode = MODE_DEFAULT;

    if(send_msg(socketfd, &mode_msg) == -1){
        perror("errore nell'invio della modalità di gioco");
        goto chiusura;
    }

    if(piazzamento_navi(socketfd) == -1){
        printf(CIANO"[BOT] Errore nell'invio della formazione al server\n"RESET);
        goto chiusura;
    }

    printf(CIANO"\n[BOT] Formazione inviata\n[*] In attesa che tutti i giocatori siano pronti...\n"RESET);
    fflush(stdout);

    azioni pck;
    bool vivo = true;

    while(vivo && !shutdown_flag){
        memset(&pck, 0, sizeof(pck));
        if(recv_msg(socketfd, &pck) == -1){
            if(!shutdown_flag) printf(CIANO"[BOT] Connessione al server
persa\n"RESET);
            break;
        }

        switch(pck.type){
            case INFO:
                if(pck.player_id != mio_id) { avversario_id = pck.player_id; }
                break;

            case TURN:
                if(pck.player_id == mio_id){
                    azioni mossa;
                    memset(&mossa, 0, sizeof(mossa));
                    mossa.player_id = mio_id;

```

```

        mossa.type = MOVE;

        selezione_prossima_mossa(&mossa.target_id, &mossa.x,
&mossa.y);

        if(send_msg(socketfd, &mossa) == -1){
            printf(CIANO"[BOT] Impossibile inviare la
mossa al server\n"RESET);
            goto chiusura;
        }
    }
    break;

case HIT:
case MISS:
    if (pck.player_id == mio_id) {
        tentativi[pck.x][pck.y] = true;

        if (pck.type == HIT) {
            if(pck.affondata < SHIP_NUMBER){
                inseguimento = false;
                direzione_colpi = -1;
                direzione_tentata = -1;
                riprovato_verso_opposto = false;
                break;
            }
            if (!inseguimento) {
                inseguimento = true;
                direzione_colpi = -1;
                direzione_tentata = -1;
                colpo_iniziale_x = pck.x;
                colpo_iniziale_y = pck.y;
                ultimo_colpo_x = pck.x;
                ultimo_colpo_y = pck.y;
            } else if (direzione_colpi == -1) {
                direzione_colpi = direzione_tentata;
                direzione_tentata = -1;
                ultimo_colpo_x = pck.x;
                ultimo_colpo_y = pck.y;
            } else {
                ultimo_colpo_x = pck.x;
                ultimo_colpo_y = pck.y;
            }
        }
    }
    break;
case ELIMINATED:
    if(pck.target_id == mio_id){
        printf(CIANO"[BOT] Sono stato eliminato\n"RESET);
        fflush(stdout);
        // chiudo -> inutile rimanere come spettatore
        vivo = false;
    }
    break;

case WIN:
    if(pck.player_id == mio_id) printf(CIANO"[BOT] Ho
vinto\n"RESET);

    else printf(CIANO"[BOT] Ha vinto il player con id %d\n"RESET,
pck.player_id);

```

```

        fflush(stdout);
        vivo = false;
        break;
    default:
        break;
    }
}

chiusura:

    if(shutdown_flag){
        printf(CIANO"\n[BOT] Chiusura forzata rilevata\n"RESET);
        fflush(stdout);
    }

    if(socketfd >= 0) close(socketfd);
    return 0;
}

int connetti(char *ip, unsigned int porta, struct sockaddr_in *server_addr){
    memset(server_addr, 0, sizeof(struct sockaddr_in));
    server_addr->sin_family = AF_INET;
    server_addr->sin_port = htons(porta);

    inet_aton(ip, &server_addr->sin_addr);

    int socketfd = socket(AF_INET, SOCK_STREAM, 0);

    if(socketfd == -1){
        perror("Errore nell'apertura del socket di connessione");
        return -1;
    }

    if(connect(socketfd, (struct sockaddr *)server_addr, sizeof(struct sockaddr_in)) ==
-1){
        perror("Errore nella connessione al server");
        close(socketfd);
        return -1;
    }

    return socketfd;
}

void selezione_prossima_mossa( uint8_t *target, uint8_t *x_out, uint8_t *y_out){
    *target = (uint8_t)avversario_id;
    int delta_riga[4] = {0,0,1,-1}; /* 0,E,S,N */
    int delta_colonna[4] = {-1,1,0,0};
    int x,y;
    if(inseguimento == false) ricerca(x_out, y_out);
    else if(direzione_colpi == -1){
        for ( int i = 0;i<4;i++){
            x = colpo_iniziale_x + delta_riga[i];
            y = colpo_iniziale_y + delta_colonna[i];
            if (cella_papabile(x,y)){
                tentativi[x][y] = true;
                *x_out = x;
                *y_out = y;
                direzione_tentata = i;
                return;
            }
        }
    }
}

```

```

        }
    }
    inseguimento = false;
    ricerca(x_out,y_out);}
else {
    x = ultimo_colpo_x + delta_riga[direzione_colpi];
    y = ultimo_colpo_y + delta_colonna[direzione_colpi];
    if (!cella_papabile(x,y) && !riprovato_verso_opposto){
        if(direzione_colpi%2 == 0) { direzione_colpi += 1;} /* serve ad
invertire la direzione con cui il bot spara */
        else { direzione_colpi -= 1;}
        x = ultimo_colpo_x + delta_riga[direzione_colpi];
        y = ultimo_colpo_y + delta_colonna[direzione_colpi];
        riprovato_verso_opposto = true;

        if(!cella_papabile(x,y)){
            inseguimento = false;
            ricerca(x_out,y_out);
            return;
        }

        tentativi[x][y] = true;
        *x_out = x;
        *y_out = y;
        return;
    }
    else if (!cella_papabile(x,y) && riprovato_verso_opposto) {
        inseguimento = false;
        riprovato_verso_opposto = false;
        ricerca(x_out,y_out);
    }
    else {
        tentativi[x][y] = true;
        *x_out = x;
        *y_out = y;
        return;
    }
}
}

void ricerca(uint8_t *x_out, uint8_t *y_out) {
    int x,y;
    int liberi = 0;

    for(int i=0; i<GRID_SIZE; i++){
        for(int j = 0; j<GRID_SIZE; j++){
            if(!tentativi[i][j]) liberi++;
        }
    }

    if(libri == 0) memset(tentativi, 0, sizeof(tentativi)); // ripulisco se sono tutte
provate

    do {
        x = rand() % GRID_SIZE;
        y = rand() % GRID_SIZE;
    } while (tentativi[x][y]);
    *x_out = x;
    *y_out = y;
    tentativi[x][y] = true;
}

```

```

}

bool cella_papabile(int x,int y) {
    if ( x >= GRID_SIZE || x < 0 || y >= GRID_SIZE || y < 0 ) return false;
    else if ( tentativi[x][y] == true) return false;
    else return true;
}

int piazzamento_navi (int socket) {
    posizionamento posizioni_navi[SHIP_NUMBER];
    int x, y;
    char orientazione;
    bool occupata[GRID_SIZE][GRID_SIZE] = {false}, valid; // griglia temporanea per capire
dove ho messo le navi
    char direzioni[4] = { 'N', 'S' , 'E', 'O' };
    for (int i = 0; i < SHIP_NUMBER; i++ ) {
        do {
            x = rand() % GRID_SIZE + 1;
            y = rand() % GRID_SIZE + 1;
            orientazione = direzioni[rand() % 4];
            valid = validazione(occupata, x - 1, y - 1, orientazione,
ship_type[i].size);
        } while (!valid);

        posizioni_navi[i].index = i;
        posizioni_navi[i].x = x-1;
        posizioni_navi[i].y = y-1;
        posizioni_navi[i].orientation = orientazione;

        printf(CIANO"[BOT] Nave %s posizionata in (%d,%d) con orientamento %c\n"RESET,
ship_type[i].name, x, y, orientazione);
        fflush(stdout);
    }

    if(invio_navi(socket, posizioni_navi) == -1) return -1;

    return 0;
}

int invio_navi(int socket, posizionamento *navi){
    for(int i = 0; i< SHIP_NUMBER; i++){
        posizionamento p;
        memset(&p, 0, sizeof(p)); //pulisco prima di inviare
        p.index = navi[i].index;
        p.x = navi[i].x;
        p.y = navi[i].y;
        p.orientation = navi[i].orientation;
        if(writen(socket, &p, sizeof(posizionamento)) != sizeof(posizionamento)){
            return -1;
        }
    }
    return 0;
}

// la validazione viene poi rifatta dal server
bool validazione( bool board[GRID_SIZE][GRID_SIZE], int x, int y, char orientazione, int
dimensione_nave ) {
    // la funzione si occupa di controllare e validare il piazzamento andando a
controllare i limiti di griglia

```

```

        // marcando quale posizione risulta occupata, non è il controllo ufficiale ma serve
solo in fase di posizionamento
        // il vero controllo poi lo rifarà anche il server.
        int delta_riga, delta_colonna, r, c;

        if ( orientazione == 'N' ) { delta_riga = -1; delta_colonna = 0;}
        else if ( orientazione == 'S' ) { delta_riga = 1; delta_colonna = 0;}
        else if ( orientazione == 'E' ) { delta_riga = 0; delta_colonna = 1;}
        else if ( orientazione == 'O' ) { delta_riga = 0; delta_colonna = -1;}
        else return false;
        for ( int i = 0; i < dimensione_nave; i++ ) {
            r = x + i * delta_riga;
            c = y + i * delta_colonna;
            if ( r < 0 || r >= GRID_SIZE || c < 0 || c >= GRID_SIZE ) {
                return false;
            } else if ( board[r][c] == true ) return false;
        }

        for ( int i = 0; i < dimensione_nave; i++ ) {
            r = x + i * delta_riga;
            c = y + i * delta_colonna;
            board[r][c] = true;
        }
        return true;
    }

void gestore(int sig){
    shutdown_flag = 1;
}

```

gui.h

```

#ifndef GUI_H
#define GUI_H
#include "../protocollo/protocollo.h"
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <stdint.h>
#define BOARD_BUFFER_SIZE 4096
// codici ANSI per i colori
#define ROSSO      "\x1b[31m"
#define VERDE      "\x1b[32m"
#define GIALLO     "\x1b[33m"
#define BLU        "\x1b[34m"
#define CIANO      "\x1b[36m"
#define BIANCO     "\x1b[37m"
#define RESET      "\x1b[0m" // colore di default del terminale
#define CLEAN      "\033[H\033[J" // comando ANSI per ripulire

#define ACQUA      CIANO
#define NAVE        BIANCO
#define COLPITO     ROSSO
#define MANCATO     GIALLO

extern char grid[GRID_SIZE][GRID_SIZE];
extern char target_grids[MAX_PLAYER + 1 ][GRID_SIZE][GRID_SIZE];
extern uint8_t targetId;

```

```

extern volatile sig_atomic_t shutdown_flag;

void gui_init(bool flag);
void init_board();
void init_target_board(uint8_t id);
void addboat(int x, int y, char orientazione, uint8_t size);
void cleanup_board(char grid[GRID_SIZE][GRID_SIZE]);
void server_connected(char *ip, unsigned int port, int id);
int game_mode();
void bersagli(uint8_t id, char *username);
void turno(void);
void ricezione_mossa(azioni *buff);
void errore_invio_mossa(void);
void non_mio_turno(uint8_t id);
void colpito(void);
void miss(void);
void spettatore(azioni *pyl);
void s_eliminato(void);
void eliminato(int id);
void s_vittoria(void);
void vittoria(uint8_t id);
void connection_lost(void);
void draw_grids();
void clean_screen();
void close_game();
void welcomeback(char *buff);
void invalid_input(char riga, int colonna, char orientamento);
void posizionamento_ok(const char *nave, char riga, int colonna, char orientazione);
void chiusura_forzata(void);
void waiting_player(void);
void connection_lost_fallback(const char *ip_buf, unsigned int port);
void mod_ospite();
void nave_affondata(const char *nave, uint8_t id);
void s_nave_affondata(const char *nave, uint8_t id);
void discovery_server_errore(const char *pname);
void sintassi_corretta(const char *pname);
void invalid_port(void);
void server_not_found(void);
void connection_error(void);
void invalid_ip(const char *ip);
void inserisci_username(size_t max);
void username_too_long(size_t len);
void inserisci_coordinate(const char *nave, uint8_t size);
void invalid_placement(void);
void discovery_req_sent(void);
void server_found(const char *ip, unsigned int port);
void failed_attempt(int i);
void sig_received(void);
void server_not_found_attempt(void);
void mostra_flotta(void);

void fflush_stdin(void); // serve per pulire il buffer di input, così da evitare che rimangano
caratteri in stdin, visto che fflush(stdin) non esiste, lo creo io

#endif

```

gui.c

```
#include "gui.h"
#include "../protocollo/protocollo.h"
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>
#include <ctype.h>

char grid[GRID_SIZE][GRID_SIZE];
char target_grids[MAX_PLAYER + 1][GRID_SIZE][GRID_SIZE]; // MAX_PLAYER + 1 perchè gli id partono da 1
static bool grigliaN_inizializz[MAX_PLAYER + 1] = {false}; // tiene traccia di quale griglia è inizializzata o meno
static bool ingame = false;
static char esito[128] = ""; // mi serve perchè non riuscivo mai a vedere le notifiche di HIT a schermo, così è sicuro che vengono stampate perchè è direttamente draw_grids() a stampare le notifiche
uint8_t targetId = 0;

typedef struct bersagli{
    uint8_t id;
    char nome[USERNAME];
    struct bersagli *next;
}nemici;

static unsigned int nemici_number = 0;
nemici *target = NULL;

void gui_init(bool flag){
    clean_screen();

    if(flag){
        printf(GIALLO " [!] Modalità connessione automatica attiva. \n [*] Startup in corso...\n" RESET);
    }

    printf(BLU "=====\\n");
    printf("                BATTAGLIA NAVALE - CLIENT v1.0                \\n");
    printf("=====\\n"RESET);
    printf(" Progetto Sistemi Operativi - A.A. 2025/2026                \\n");
    printf(" Realizzato da:                \\n");
    printf(VERDE "  [*] Lorenzo Tarantino (TenseNotFound)                \\n");
    printf("  [*] Leonardo (lrcicalini)                \\n"RESET);
    printf(" Link alla repository:                \\n");
    printf(BLU "  [*] https://github.com/TenseNotFound/Progetto-Sistemi-Operativi                \\n");
    printf("=====\\n\\n"RESET);

    printf("Benvenuto nel gioco battaglia navale!\\n");
    printf(GIALLO " [*] In attesa della connessione con il server...\\n\\n"RESET);

    fflush(stdout);
}

void server_connected(char *ip, unsigned int port, int id){
    printf(" [*] Il tuo ID e': " VERDE "%d\\n" RESET, id);
```



```

        fflush(stdout);
    }

void init_board(){
    cleanup_board(grid);
    targetId = 0;
    for(int i = 0; i<=MAX_PLAYER; i++){
        grigliaN_inizializz[i] = false;
    }
    esito[0] = '\0';
}

void cleanup_board(char grid[GRID_SIZE][GRID_SIZE]){
    for(int i = 0; i<GRID_SIZE; i++){
        for(int j = 0; j<GRID_SIZE; j++){
            grid[i][j] = '~';
        }
    }
}

void init_target_board(uint8_t id){
    if(id > MAX_PLAYER) return;
    if(!grigliaN_inizializz[id]) {
        cleanup_board(target_grids[id]);
        grigliaN_inizializz[id] = true;
    }
}

static void draw_board(char griglia[GRID_SIZE][GRID_SIZE]) {
    char buffer[BOARD_BUFFER_SIZE];
    size_t offset = 0;
    int scritto = 0;
    char temp[16];

    int col_width = snprintf(temp, sizeof(temp), "%d", GRID_SIZE);
    scritto = snprintf(buffer + offset, sizeof(buffer) - offset, "    ");
    offset += (scritto > 0) ? (size_t) scritto : 0;

    for (int i = 0; i<GRID_SIZE; i++){
        scritto = snprintf(buffer + offset, sizeof(buffer) - offset, "%-*d", col_width, i+1);
        offset += (scritto > 0) ? (size_t) scritto : 0;
    }

    scritto = snprintf(buffer + offset, sizeof(buffer)- offset, "\n"); // mando a capo dopo la
    riga dell'intestazione
    offset += (scritto > 0) ? (size_t) scritto : 0;

    for(int i = 0; i<GRID_SIZE; i++){

        scritto = snprintf(buffer + offset, sizeof(buffer) - offset, " %c ", 'A' + i);
        offset += (scritto > 0) ? (size_t) scritto : 0;
        for(int j = 0; j<GRID_SIZE; j++){
            scritto = snprintf(buffer+offset, sizeof(buffer) - offset, "|%s%c%s",
                                griglia[i][j] == 'N' ? NAVE :
                                griglia[i][j] == 'X' ? COLPITO :
                                griglia[i][j] == 'O' ? MANCATO :
                                ACQUA,
                                griglia[i][j],
                                RESET);
            offset += (scritto > 0) ? (size_t) scritto : 0;
        }
    }
}

```

```

    }

    scritto = snprintf(buffer + offset, sizeof(buffer) - offset, "|\n");
    offset += (scritto > 0) ? (size_t) scritto : 0;
}

scritto = snprintf(buffer + offset, sizeof(buffer) - offset, "\n");
offset += (scritto > 0) ? (size_t) scritto : 0;

fputs(buffer, stdout);
fflush(stdout);
}

void draw_grids(){
    printf(VERDE"\n\n--- LA TUA FLOTTA ---\n"RESET);
    draw_board(grid);

    printf(ROSSO"\n--- RADAR NEMICO ---\n"RESET);
    if(targetId == 0) {
        printf(GIALLO" [!] Nessun nemico ancora selezionato\n"RESET);
    } else {
        printf(" [*] Radar del player id:%d\n", targetId);
        draw_board(target_grids[targetId]);
    }

    if(esito[0] != '\0'){
        printf("\n"CIANO"==>"RESET"%s"CIANO"<==="RESET"\n", esito);
    }
    fflush(stdout);
}

void clean_screen(){
    printf(CLEAN); //codice ANSI per pulire lo schermo
}

void close_game(){
    if(ingame) printf(VERDE"\n [!] Grazie per aver giocato! Arrivederci!\n"RESET);
    else printf(GIALLO"\n [!] Chiusura del client\n"RESET);
    fflush(stdout);
}

void addboat(int x, int y, char orientazione, uint8_t size){
    int riga_off = 0;
    int col_off = 0;

    if (orientazione == 'N') {
        riga_off = -1; // alto
    } else if (orientazione == 'S') {
        riga_off = 1; // basso
    } else if (orientazione == 'E') {
        col_off = 1; // destra
    } else if (orientazione == 'O') {
        col_off = -1; // sinistra
    }

    for (int i = 0; i < size; i++) {
        int r = x + (i * riga_off);
        int c = y + (i * col_off);

        if (r >= 0 && r < GRID_SIZE && c >= 0 && c < GRID_SIZE) {

```

```

        grid[r][c] = 'N';
    }
}

int game_mode(){

    char input[BUFFER_SIZE];

    printf(BLU "\n [*] Scegli la modalità di gioco! \n"RESET);
    printf(VERDE"    [1] 1v1 \n"RESET);
    printf(GIALLO"    [2] Default [Premi INVIO o qualsiasi altro tasto]\n"RESET);
    fflush(stdout);

    if(fgets(input, sizeof(input), stdin) == NULL){
        return MODE_DEFAULT;
    }

    input[strlen(input)-1] = '\\0';

    if(strcmp(input, "1") == 0){
        return MODE_1V1;
    }
    return MODE_DEFAULT;
}

void connection_lost(void){

    printf(ROSSO"\n [!] Connessione al server persa\n"RESET);
    fflush(stdout);
}

void bersagli(uint8_t id, char *username){
    // serve per aggiungere i nuovi nemici, si usa una lista a puntatori perchè a priori non
    // so quanti bersagli ci sono
    nemici *nemico = malloc(sizeof(nemici));

    if(nemico == NULL){
        perror("Errore nella malloc per allocare il nuovo nemico");
        return;
    }

    nemico->id = id;
    strcpy(nemico->nome, username);

    nemico->next = target;
    target = nemico;
    nemici_number++;
}

static void free_bersagli(void){
    nemici *curr = target, *temp;
    while(curr != NULL) {
        temp = curr;
        curr = curr->next;
        free(temp);
    }
    target = NULL;
    nemici_number = 0;
}

```

```

static bool bersaglio_valido(int id){
    for(nemici *curr = target; curr != NULL; curr = curr->next){
        if(curr->id == id) return true;
    }
    return false;
}

void turno() {
    printf(VERDE"\n===== \n");
    printf("          [*] È IL TUO TURNO\n");
    printf("===== \n"RESET);

    printf(ROSSO"\n----- BERSAGLI DISPONIBILI ----- \n"RESET);
    nemici *curr = target;
    int i = 0;
    while(curr != NULL) {
        printf(BLU "          [%d] [ID: %d] %s\n"RESET, i+1, curr->id, curr->nome);
        curr = curr->next;
        i++;
    }
    printf(ROSSO "----- \n"RESET);
}

void ricezione_mossa(azioni *mossa) {
    int bersaglio, y;
    char x_in[2];
    bool mossa_valida = false;

    mossa->type = MOVE;
    int letti;
    while (!mossa_valida) {
        if(nemici_number == 1 && target != NULL){
            bersaglio = target->id;
        } else {
            printf("\n [*] Inserisci l'ID del giocatore da colpire: ");
            fflush(stdout);
            if((letti = scanf("%d", &bersaglio)) == EOF){
                // se viene chiuso l'input il gioco non funziona più -> chiudo
                shutdown_flag = 1;
                free_bersagli();
                return;
            }
            if(letti != 1 || !bersaglio_valido(bersaglio)) {
                printf(GIALLO" [!] Inserisci un id tra quelli disponibili\n"RESET);
                fflush(stdout);
                fflush_stdin();
                continue;
            }
        }

        if(bersaglio == mossa->player_id){
            printf(GIALLO" [!] Non puoi fare fuoco a te stesso!\n"RESET);
            fflush(stdout);
            fflush_stdin();
            continue;
        }

        while(!mossa_valida){
            printf("Inserisci le coordinate: <A-J> <1-10>: ");

```

```

    letti = scanf("%1s %d", x_in, &y);
    if(shutdown_flag || letti == EOF){
        shutdown_flag = 1;
        free_bersagli();
        return;
    }

    if(letti != 2){
        printf(GIALLO" [!] Inserisci delle coordinate valide !\n"RESET);
        fflush_stdin();
        fflush(stdout);
        continue;
    }

    fflush_stdin();

    y = y - 1;
    int x = toupper(x_in[0]) - 'A'; // ASCII -> 'A' = 65

    if (x >= 0 && x < GRID_SIZE && y >= 0 && y < GRID_SIZE) {
        init_target_board((uint8_t)bersaglio);
        if (target_grids[bersaglio][x][y] == 'X' || target_grids[bersaglio][x][y] ==
'0') {
            printf(GIALLO" [!] Hai già sparato in queste coordinate!\n"RESET);
            fflush(stdout);
        } else {
            mossa->target_id = bersaglio;
            mossa->x = x;
            mossa->y = y;
            targetId = (uint8_t)bersaglio;
            mossa_valida = true;
        }
    } else {
        printf(ROSSO" [!] Coordinate non valide. Devi inserire Lettera (A-J) e Numero
(1-10).\n"RESET);
        fflush(stdout);
    }
}
}
free_bersagli();
}

void errore_invio_mossa(void) {
    printf(ROSSO"\n [!] Errore: Impossibile inviare la mossa al server.\n"RESET);
    fflush(stdout);
}

void non_mio_turno(uint8_t id) {
    printf(GIALLO"\n [*] È il turno del giocatore %u. In attesa... \n"RESET, id);
    fflush(stdout);
}

void colpito() {
    snprintf(esito, sizeof(esito), VERDE" [+] BERSAGLIO COLPITO! "RESET);
    fflush(stdout);
}

void miss() {
    snprintf(esito, sizeof(esito), GIALLO" [-] Mancato !"RESET);
    fflush(stdout);
}

```

```

}

void nave_affondata(const char *nave, uint8_t id){
    snprintf(esito, sizeof(esito), VERDE" [!] Hai affondato %s (%d)! "RESET, nave, id);
    fflush(stdout);
}

void spettatore(azioni *pck) {
    char esito_str[20];
    if (pck->type == HIT) strcpy(esito_str, VERDE"COLPITO"RESET);
    else strcpy(esito_str, GIALLO"MANCATO"RESET);

    printf(BLU"\n [*] Il giocatore %d ha sparato al giocatore %d in %c%d: %s!\n"RESET,
           pck->player_id, pck->target_id, pck->x + 'A', pck->y + 1, esito_str);
    fflush(stdout);
}

void s_eliminato() {
    printf(ROSSO"\n=====\\n");
    printf(" [!] SEI STATO ELIMINATO! LA TUA FLOTTA È AFFONDATA [!]\n");
    printf("=====\\n"RESET);
    printf(CIANO"[*] MODALITÀ SPETTATORE ATTIVA\\n"RESET);
    printf(CIANO"[*] Sei ora uno spettatore\\n"RESET);
    printf(GIALLO"[*] Rimani in attesa per guardare il resto della partita.\\n"RESET);
    fflush(stdout);
}

void eliminato(int id) {
    printf(ROSSO"\n [!] IL GIOCATORE %d È STATO ELIMINATO!\\n"RESET, id);
    fflush(stdout);
}

void s_vittoria() {
    printf(VERDE"\n=====\\n");
    printf("          [*] VITTORIA! [*]\n");
    printf("=====\\n"RESET);
    fflush(stdout);
}

void vittoria(uint8_t id) {
    printf(VERDE"\n=====\\n");
    printf("          [*] LA PARTITA È CONCLUSA! HA VINTO IL GIOCATORE %u [*]\n", id);
    printf("=====\\n"RESET);
    fflush(stdout);
}

void fflush_stdin(void){
    int c;
    while((c = getchar()) != '\\n' && c != EOF);
}

void welcomeback(char *buff){
    printf(VERDE" [*] Bentornato %s! \\n"RESET, buff);
    fflush(stdout);
}

void invalid_input(char riga, int colonna, char orientamento){
    printf(ROSSO" [!] Input non valido -> riga: '%c' colonna: %d orientamento: '%c' non
valide! \\n [*] Sintassi corretta: <A-J> <1-10> <orientamento (N,S,E,O/W)>\\n"RESET, riga,
colonna, orientamento);
}

```

```

        fflush(stdout);
    }

void posizionamento_ok(const char *nave, char riga, int colonna, char orientazione){
    printf(VERDE" [*] Nave %s posizionata in (%c,%d) con orientamento %c\n"RESET, nave, riga,
colonna, orientazione);
    fflush(stdout);
}

void chiusura_forzata(void){
    printf(ROSSO"\n [!] Chiusura forzata del client, inizio routine di shutdown...\n"RESET);
    fflush(stdout);
}

void waiting_player(void){
    ingame = true;
    printf(GIALLO"\n [*] In attesa che tutti i giocatori siano pronti...\n"RESET);
    fflush(stdout);
}

void connection_lost_fallback(const char *ip_buf, unsigned int port){
    printf(GIALLO" [*] Connessione a %s:%u fallita, provo il discovery automatico...\n"RESET,
ip_buf, port);
    fflush(stdout);
}

void mod_ospite(void){
    printf(GIALLO" [!] Modalità ospite attivata\n"RESET);
    fflush(stdout);
}

void s_nave_affondata(const char *nave, uint8_t id){
    snprintf(esito, sizeof(esito), ROSSO" [!] Il giocatore con id %d ha affondato la nave %s
della tua flotta!"RESET, id, nave);
}

void discovery_server_errore(const char *pname){
    printf(ROSSO" " [!] Errore nel discovery del server, inserisci manualmente ip e porta\n
Sintassi: %s <IP> <port>\n"RESET, pname);
    fflush(stdout);
}

void sintassi_corretta(const char *pname){
    printf(GIALLO" " [!] Sintassi corretta: %s <IP> <port>\n"RESET, pname);
    fflush(stdout);
}

void invalid_port(){
    printf(GIALLO" " [!] Inserisci un numero di porta valido nel range 5000-65535, provo
ora il discovery automatico...\n"RESET);
    fflush(stdout);
}

void server_not_found(){
    printf(ROSSO" " [!] Errore: impossibile trovare un server\n" RESET);
    fflush(stdout);
}

void connection_error(){
    printf(ROSSO" [!] Impossibile stabilire una connessione con il server\n"RESET);
}

```

```

        fflush(stdout);
    }

void invalid_ip(const char *ip){
    printf(GIALLO" [!] Indirizzo ip non valido: %s\n"RESET, ip);
    fflush(stdout);
}

void inserisci_username(size_t max){
    printf("\n [*] Inserisci il tuo username (max %zu caratteri): ", max);
    fflush(stdout);
}

void username_too_long(size_t len){
    printf(GIALLO" [!] Inserisci un username di massimo %zu caratteri!\n"RESET, len);
    fflush(stdout);
}

void inserisci_coordinate(const char *nave, uint8_t size){
    printf(" [*] Stai piazzando la nave %s (dimensione %u)(es: A 1 E):\n", nave, size);
    fflush(stdout);
}

void invalid_placement(){
    printf(GIALLO" [!] Posizionamento non valido: la nave esce dalla griglia o si sovrappone
con un'altra nave. Riprova.\n"RESET);
    fflush(stdout);
}

void discovery_req_sent(){
    printf(GIALLO" [*] Richiesta di discovery inviata con successo (broadcast). \n In attesa
di riscontro dal server...\n"RESET);
    fflush(stdout);
}

void server_found(const char *ip, unsigned int port){
    printf(VERDE" [*] Server trovato con successo!\n [*] In ascolto su %s:%u\n"RESET, ip,
port);
    fflush(stdout);
}

void failed_attempt(int i){
    printf(GIALLO" [!] Tentativo %d fallito (Timeout), riprovo...\n"RESET, i + 1);
    fflush(stdout);
}

void sig_received(void){
    printf(GIALLO" [*] Ricevuta segnalazione, interrompo... \n"RESET);
    fflush(stdout);
}

void server_not_found_attempt(){
    printf(ROSSO" [*] Nessun server trovato in rete dopo %d tentativi.\n"RESET, TENTATIVI);
    fflush(stdout);
}

void mostra_flotta(){
    printf(VERDE"\n\n\n--- LA TUA FLOTTA ---\n"RESET);
    draw_board(grid);
}

```


protocollo.h

```
#ifndef PROTOCOL_H
#define PROTOCOL_H

#include <stdint.h>
#include <string.h>
#define SHIP_NUMBER 5
#define GRID_SIZE 10
#define BACKLOG 16
#define MAX_PLAYER 32 // scelta obbligata via dimensioni fisse dei semafori, ho bisogno a priori di un semaforo con dim ≥ del numero slot di client accettati per
                        // poter accettare correttamente tutti i nuovi player
#define TIMEOUT_SHUTDOWN 10 // (secondi)
#define TIMEOUT_SOCKET 2 // (secondi) usato dal client in WINAPI per il timer di timeout per la discovery port
#define AFK_TIMEOUT 180 // (secondi) inattività AFK
#define TIMEOUT_LOBBY 30
#define DISCOVERY_PORT 9999
#define DISCOVER "DISCOVER"
#define BUFFER_SIZE 256
#define TENTATIVI 20 // per la ricerca della porta
#define NAME_LEN 16 // (user side) quanti caratteri -> MODIFICARE QUI E NON USERNAME
#define USERNAME (NAME_LEN + 1) // da usare nel codice
#define NESSUNA_NAVE SHIP_NUMBER // valore massimo non valido per comunicare l'indice della nave affondata

typedef enum game_info{ // serve poi per capire che mossa/azione è stata fatta/compiuta/subita sul client
    WELCOME, JOIN, TURN, MOVE, HIT, MISS, ELIMINATED, WIN, MODE, INFO
}game_info;

typedef enum mode {
    MODE_DEFAULT,
    MODE_1V1
} mode;

// serve per la compattazione dei dati che trasmetto in rete -> riduzione di payload -> meno overhead complessivo -> portabilità tra piattaforme
#pragma pack(push, 1)
typedef struct azioni{ // mi definisce la mossa
    game_info type; // specifico il tipo di azione
    uint8_t player_id;
    uint8_t target_id;
    uint8_t x,y; // posizione nave da colpire
    mode gamemode;
    char username[USERNAME];
    uint8_t affondata; // indice della nave affondata
} azioni;

typedef struct posizionamento{ // mi definisce la nave
    uint8_t index; //indice in ship_type
    uint8_t x,y; //coordinate nave
    char orientation; //orientamento della nave (N,S,E,O)
}posizionamento;
#pragma pack(pop)

typedef struct navi{
    const char *name;
    uint8_t size; // così un solo byte, con numeri da 0-255 rispetto a 4 byte
```

```

} nave;

extern const nave ship_type[SHIP_NUMBER]; // così non creo 3 blocchi di memoria identica
// serve const così non ci sono problemi di
scrittura quindi sincronizzazione
// è read-only

#endif

```

protocollo.c

```

#include "protocollo.h"

const nave ship_type[SHIP_NUMBER] = { // spiegazione in protocollo.h
    {"Portaerei", 5},
    {"Corazzata", 4},
    {"Incrociatore", 3},
    {"Sottomarino", 3},
    {"Cacciatorpediniere", 2}
};

```

utils.h

```

/*
    FUNZIONI RIPETUTE ED IDENTICHE TRA sever.c E client.c
    QUI SONO CONTENUTE LE FIRME DELLE FUNZIONI
*/

#ifndef UTILS_H
#define UTILS_H
#include "../protocollo/protocollo.h"
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <signal.h>
#include <sys/types.h>
#include <string.h>

#ifdef _WIN32
    #include <winsock2.h>
    #include <ws2tcpip.h>
#else
    #include <sys/socket.h>
    #include <netinet/in.h>
    #include <netdb.h>
    #include <arpa/inet.h>
    #include <unistd.h>
#endif

extern volatile sig_atomic_t shutdown_flag; // mi serve perchè sia client.c che server.c usano
un procedimento di sblocco da queste funzioni basato su shutdown_flag impostato dal gestore ad
1
// extern perchè definita in client.c e server.c

#ifdef _WIN32
    int recv_msg(SOCKET fd, azioni *msg); // IDENTICA -> serve per la ricezione di messaggi
    int send_msg(SOCKET fd, azioni *msg); // IDENTICA -> serve per la spedizione di messaggi
    SSIZE_T readn(SOCKET fd, void *buf, size_t n); // serve per controllare l'avvenuta lettura
di tutti i dati in rete IDENTICA

```

```

        ssize_t writen(SOCKET fd, const void *buf, size_t n); // serve per controllare l'avvenuta
scrittura di tutti i dati in rete    IDENTICA
#else
    int recv_msg(int fd, azioni *msg); // IDENTICA -> serve per la ricezione di messaggi
    int send_msg(int fd, azioni *msg); // IDENTICA -> serve per la spedizione di messaggi
    ssize_t readn(int fd, void *buf, size_t n); // serve per controllare l'avvenuta lettura di
tutti i dati in rete    IDENTICA
    ssize_t writen(int fd, const void *buf, size_t n); // serve per controllare l'avvenuta
scrittura di tutti i dati in rete    IDENTICA
#endif

#endif

```

utils.c

```

/*
FUNZIONI RIPETUTE ED IDENTICHE TRA sever.c E client.c
QUI SONO CONTENUTE LE DICHIARAZIONI DELLE FUNZIONI
*/

#include "../protocollo/protocollo.h"
#include "utils.h"
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <string.h>
#include <errno.h>
#include <signal.h>
#include <stdint.h>
#include <unistd.h>
#ifdef _WIN32
    #include <arpa/inet.h>
#else
    #include <windows.h>
#endif

#ifdef _WIN32
ssize_t readn(int fd, void *buf, size_t n){
    /*
    QUESTA FUNZIONE SERVE PER IL CONTROLLO
    DELL'AVVENUTA LETTURA DI TUTTI I DATI IN RETE
    SE NE LEGGO DI MENO CONTINUO A LEGGERE ESCLUDENDO I VARI CASI
    */
    size_t left = n;
    char *p = (char *)buf;

    while(left > 0){
        ssize_t r = read(fd, p, left);
        if(r < 0){
            if(errno == EINTR){ // lettura bloccata da segnalazione, riprovo
                if(shutdown_flag) return -1;
                continue;
            }
            return -1;
        }
        if (r == 0) return (ssize_t)(n - left); // il client ha chiuso la connessione, analogo
chiusura PIPE
        left -= (size_t)r;
        p += r;
    }
}

```

```

    }
    return (ssize_t)n; // se arrivo qui ho letto tutti i byte richiesti e comunico con n
}

ssize_t writen(int fd, const void *buff, size_t n){
    /*
        QUESTA FUNZIONE SERVE PER IL CONTROLLO
        DELL'AVVENUTA SCRITTURA DI TUTTI I DATI IN RETE
        SE NE SCRIVO DI MENO CONTINUO A SCRIVERE ESCLUDENDO I VARI CASI
    */
    size_t left = n;
    const char *p = (const char *)buff;
    while(left > 0){ // > e non < perchè è unsigned e non può essere negativo
        ssize_t w = write(fd, p, left);
        if(w<0){
            if(errno == EINTR){ // scrittura bloccata da segnalazione, riprovo
                if(shutdown_flag) return -1;
                continue;
            }
            return -1;
        } if(w == 0) return (ssize_t)(n - left); // il client ha chiuso la connessione,
        analogo chiusura PIPE
        left -= (size_t)w;
        p += w;
    }
    return (ssize_t)n; // se arrivo qui ho scritto tutti i byte richiesti e comunico con n
}

#else
SSIZE_T readn(SOCKET fd, void *buf, size_t n){
    /*
        QUESTA FUNZIONE SERVE PER IL CONTROLLO
        DELL'AVVENUTA LETTURA DI TUTTI I DATI IN RETE
        SE NE LEGGO DI MENO CONTINUO A LEGGERE ESCLUDENDO I VARI CASI
    */
    size_t left = n;
    char *p = (char *)buf;

    while(left > 0){
        int r = recv(fd, p, (int)left, 0);
        if(r<0){
            if(r == SOCKET_ERROR) return -1; // errore nel socket non dovuto da segnalazione
            return -1;
        }
        if (r == 0) return (ssize_t)(n - left); // il client ha chiuso la connessione, analogo
        chiusura PIPE
        left -= (size_t)r;
        p += r;
    }
    return (ssize_t)n; // se arrivo qui ho letto tutti i byte richiesti e comunico con n
}

SSIZE_T writen(SOCKET fd, const void *buff, size_t n){
    /*
        QUESTA FUNZIONE SERVE PER IL CONTROLLO
        DELL'AVVENUTA SCRITTURA DI TUTTI I DATI IN RETE
        SE NE SCRIVO DI MENO CONTINUO A SCRIVERE ESCLUDENDO I VARI CASI
    */
    size_t left = n;
    const char *p = (const char *)buff;

```

```

while(left > 0){ // > e non < perchè è unsigned e non può essere negativo
    int w = send(fd, p, (int)left, 0);
    if(w<0){
        if(w == SOCKET_ERROR) return -1; // errore nel socket non dovuto da segnalazione
        return -1;
    } if(w == 0) return (ssize_t)(n - left); // il client ha chiuso la connessione,
analogo chiusura PIPE
    left -= (size_t)w;
    p += w;
}
return (ssize_t)n; // se arrivo qui ho scritto tutti i byte richiesti e comunico con n
}
#endif

#ifdef _WIN32
int send_msg(SOCKET fd, azioni *msg){
#else
int send_msg(int fd, azioni *msg){
#endif
    azioni packet;
    memset(&packet, 0, sizeof(packet));

    //carico i dati in rete
    packet.type = htonl(msg->type);
    packet.player_id = msg->player_id;
    packet.target_id = msg->target_id;
    packet.x = msg->x; // endianness free -> mando 1 byte e non 4, nessun problema di endiannes
    packet.y = msg->y;
    packet.affondata = msg->affondata;
    packet.gamemode = htonl(msg->gamemode);
    memcpy(packet.username, msg->username, USERNAME);
// c'è rischio che in TCP si scrivano meno byte di quelli richiesti,

#ifdef _WIN32
    SSIZE_T n = writen(fd, &packet, sizeof(azioni));
    // c'è rischio che in TCP si scrivano meno byte di quelli richiesti,
    if(n != (SSIZE_T)(sizeof(azioni))){
        return -1;
    }
#else
    ssize_t n = writen(fd, &packet, sizeof(azioni));
    // c'è rischio che in TCP si scrivano meno byte di quelli richiesti,
    if(n != (ssize_t)(sizeof(azioni))){
        return -1;
    }
#endif
    return 0;
}

#ifdef _WIN32
int recv_msg(SOCKET fd, azioni *msg){
#else
int recv_msg(int fd, azioni *msg){
#endif
    azioni packet;

#ifdef _WIN32
    SSIZE_T n = readn(fd, &packet, sizeof(azioni));
    // c'è rischio che in TCP si ricevano meno byte di quelli richiesti,

```

```

        // quindi bisogna fare un ciclo finchè non si ricevono tutti i byte
        if(n == (SSIZE_T)(sizeof(azioni))){
#else
        ssize_t n = readn(fd, &packet, sizeof(azioni));
        // c'è rischio che in TCP si ricevano meno byte di quelli richiesti,
        // quindi bisogna fare un ciclo finchè non si ricevono tutti i byte
        if(n == (ssize_t)(sizeof(azioni))){
#endif

        //scarico i dati da rete
        msg->type = ntohl(packet.type);
        msg->player_id = packet.player_id;
        msg->target_id = packet.target_id;
        msg->x = packet.x; // endianess free -> mando 1 byte e non 4, nessun problema di
endiannes
        msg->y = packet.y;
        msg->affondata = packet.affondata;
        msg->gamemode = ntohl(packet.gamemode);
        memcpy(msg->username, packet.username, USERNAME);
        msg->username[USERNAME - 1] = '\0';

        return 0;
    } else if(n == 0){
        return -1;
    }

    return -1;
}

```

Makefile

```

# Progetto Sistemi Operativi - A.A. 2025/2026
# Battaglia navale multiutente
# Lorenzo Tarantino - Leonardo Rocco Cicalini

all: server client

server:
    @gcc src/server/server.c utils/utils.c protocollo/protocollo.c -o battaglia_server
-lpthread
    @gcc src/bot/bot.c utils/utils.c protocollo/protocollo.c -o battaglia_bot
    @echo "==> Server e Bot pronti per l'esecuzione"

client:
    @gcc src/client/client.c utils/utils.c protocollo/protocollo.c gui/gui.c -o
battaglia_client
    @echo "==> Client (POSIX) pronto per l'esecuzione"

wclient:
    @gcc src/client/client.c utils/utils.c protocollo/protocollo.c gui/gui.c -o
battaglia_client.exe -lws2_32
    @echo "==> Client (WINAPI) pronto per l'esecuzione"

bot:
    @gcc src/bot/bot.c utils/utils.c protocollo/protocollo.c -o battaglia_bot
    @echo "==> Bot pronto per l'esecuzione"

```

clean:

```
@rm -f battaglia_server battaglia_client battaglia_client.exe battaglia_bot _user
@echo "==> Eseguibili rimossi correttamente"
```

help:

```
@echo "=====
@echo "                                ISTRUZIONI D'USO                                "
@echo "=====
@echo "  make all      - Compila server, client e bot per standard POSIX"
@echo "  make server   - Compila sia il server che il bot"
@echo "  make client   - Compila solo il client (POSIX)"
@echo "  make wclient  - Compila solo il client (WINAPI)"
@echo "  make bot      - Compila solo il bot"
@echo "  make clean    - Rimuove tutti gli eseguibili compilati"
@echo "  make help     - Mostra questo elenco di comandi"
@echo "=====
```